TAGORE ENGINEERING COLLEGE

RATHINAMANGALAM, CHENNAI- 600127



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS25C11-OPERATING SYSTEMS LABORATORY

Name : NITISH S

Reg. No : 412725104073

Branch : BE CSE

Year : 2025-2029

Semester: III

Anna University:: Chennai

Regulation 2025

TAGORE ENGINEERING COLLEGE

RATHINAMANGALAM, CHENNAI-600127

LABORATORY RECORD

UNIVERSITY REGISTER NO. | |

Certified that this is the bonafide record of work done by

Mr. /Ms. _____ of _____

Department in the _____ laboratory

and submitted for the University Practical Examination conducted on _____

at TAGORE ENGINEERING COLLEGE, CHENNAI-127.

Record Mark: _____

Lab In-charge **Principal** **Head of the Department**

Internal Examiner **External Examiner**

VISION

To create an environment which is conducive to produce competent Computer Science Engineers through quality education and research-oriented education and equip them for the needs of the industry and society.

MISSION

The Department strives to contribute to the expansion of knowledge in the discipline of Computer Science and Engineering by

- Adopting an efficient teaching learning process in concurrence with increasing industrial demands.
- Ensuring technical proficiency, facilitating to pursue higher studies and carry out Research &Development activities.
- Developing problem solving and analytical skills with deep knowledge in thorough understandingof basic sciences
- and Computer Science Engineering.
- Infusing managerial and entrepreneurship skills to become ethical, socially responsible, andcompetitive professionals.

PROGRAM SPECIFIC OBJECTIVES (PSOs)

1. Exhibit design and programming skills to build and automate business solutions using cutting edgetechnologies

2. Strong theoretical foundation leading to excellence and excitement towards research, to provide elegantsolutions to complex problems.

3. Ability to work effectively with various engineering fields as a team to design, build and develop system applications.

PROGRAM OUTCOMES POs

Engineering Graduates will be able to:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.

2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of theinformation to provide valid conclusions.

5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need forsustainable development.

8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and normsof the engineering practice.

9. **Individual and team work**: Function effectively as an individual, and as a member or leader indiverse teams, and in multidisciplinary settings.

10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one ‘s own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change

COURSE OUTCOMES:

At the end of this course, the students will be able to:

CO1: Describe the operating systems concepts in managing computer hardware and software resources.

CO2: Analyze operating system components and file systems .

CO3: Evaluate operating system algorithms in different computing environments.

CO4: Design operating system solutions for managing system resources in real-world scenarios.

CO5: Develop the ability to adapt to emerging trends, tools, and technologies.

GENERAL INSTRUCTIONS FOR LABORATORY CLASSES

- Students must wear ID card and shoes before entering the lab.
- Boys should wear formal shirt, pant and should “tuck in” the shirts.
- You must shutdown the system, switch off the power supply and arrange the chairs properly before leaving the lab.
- Maintain Observation Notebook and Lab Record neatly and complete them then and there after completion of each exercise.
- Maintain discipline in the lab and be silent throughout the lab session.

LIST OF EXPERIMENTS

S. No	Date	Name of the Experiment	PageNo	Marks	Sign
1		INSTALLATION OF LINUX OPERATING SYSTEM	1		
2		LINUX COMMANDS AND SHELL SCRIPTING			
3		PROCESS CREATION USING fork(), exec(), wait()			
4		MULTI-THREADING USING POSIX THREADS			
5.1		FIRST COME FIRST SERVE (FCFS) CPU SCHEDULING			
5.2		SHORTEST JOB FIRST (SJF) CPU SCHEDULING			
5.3		ROUND ROBIN (RR) CPU SCHEDULING			
5,4		PRIORITY CPU SCHEDULING			
5,5		MULTILEVEL QUEUE CPU SCHEDULING			
6.1		PRODUCER – CONSUMER PROBLEM			
6,2		READERS – WRITERS PROBLEM			

7		DEADLOCK AVOIDANCE USING BANKER'S ALGORITHM			
8.1		FIFO PAGE REPLACEMENT ALGORITHM (FIFO)			
8.2		LEAST RECENTLY USED (LRU) PAGE REPLACEMENT ALGORITHM			
8.3		OPTIMAL PAGE REPLACEMENT ALGORITHM			
9		DYNAMIC MEMORY ALLOCATION AND DEALLOCATION			
10		FILE OPERATIONS AND FILE BUFFERING			
11.1		DISK SCHEDULING ALGORITHMS (FCFS)			
11,2		SHORTEST SEEK TIME FIRST (SSTF) DISK SCHEDULING			
11.3		SCAN DISK SCHEDULING ALGORITHM			
11.4		C-SCAN DISK SCHEDULING ALGORITHM			
11.5		LOOK DISK SCHEDULING ALGORITHM			

Ex. No: 1 **INSTALLATION OF LINUX OPERATING SYSTEM**

DATE:

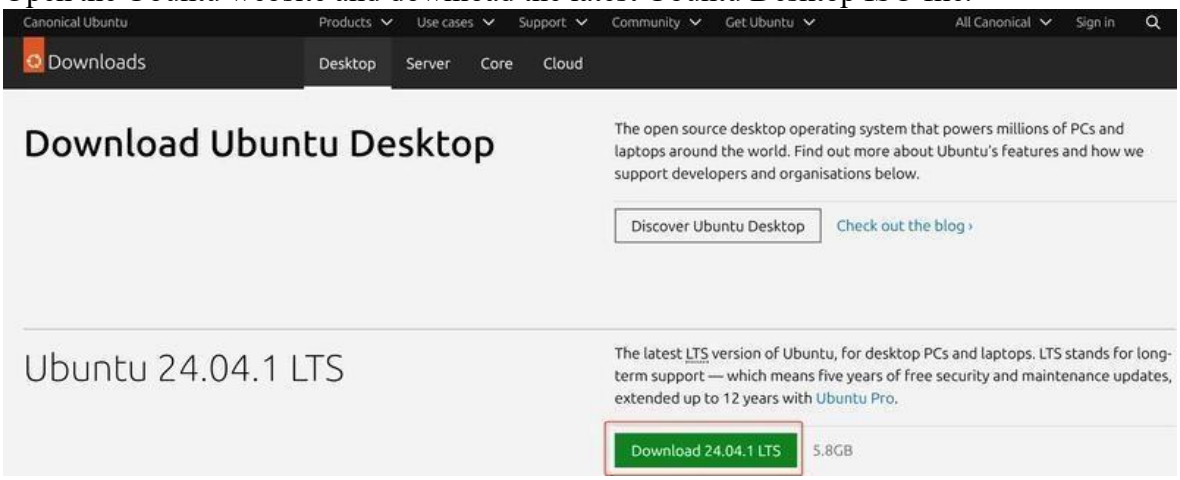
AIM

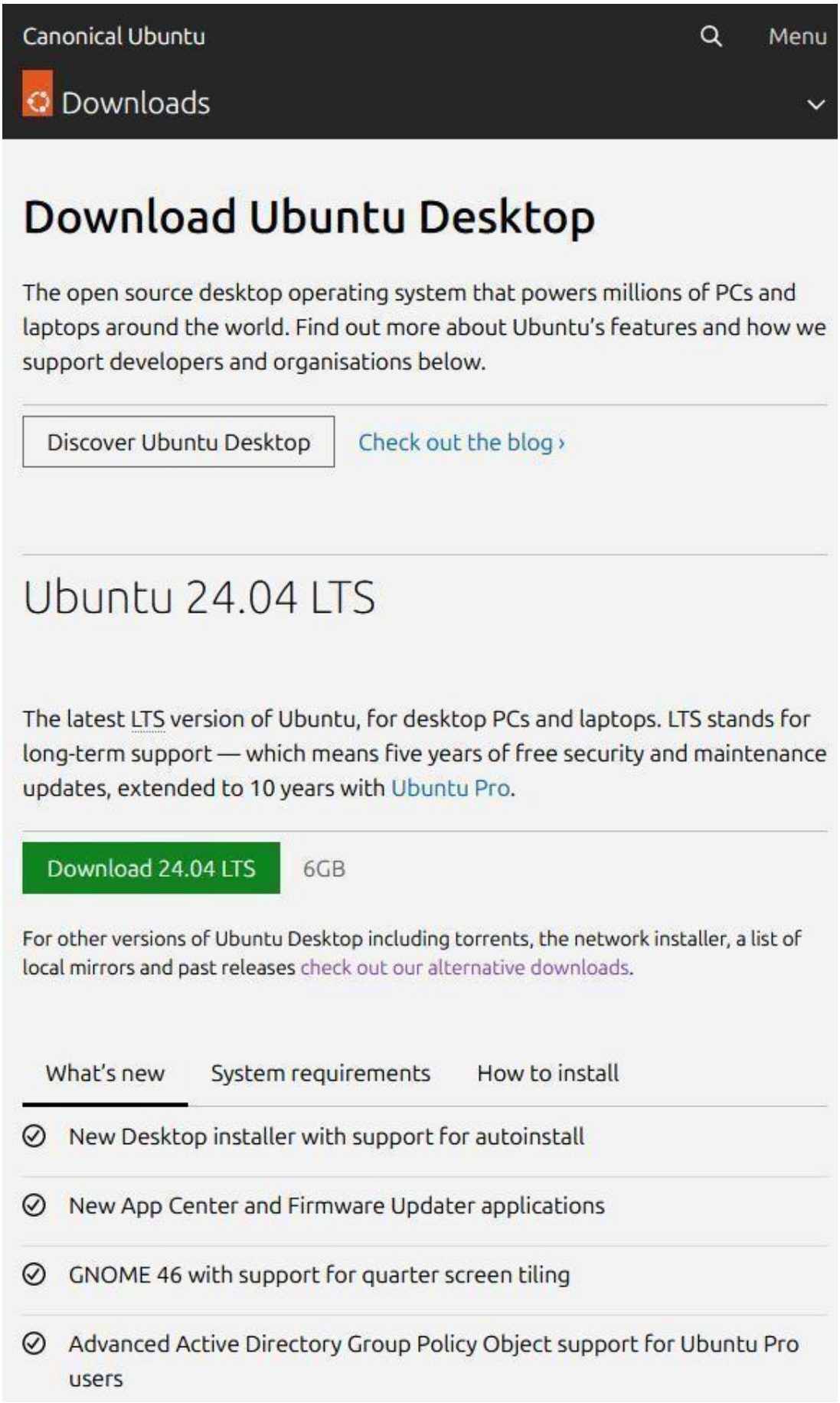
To install the Linux Operating System and become familiar with its installation process, basic configuration, and working environment.

PROCEDURE

Step 1: Download Ubuntu Linux

Open the Ubuntu website and download the latest Ubuntu Desktop ISO file.




Canonical Ubuntu
Menu
Downloads
Download Ubuntu Desktop
The open source desktop operating system that powers millions of PCs and laptops around the world. Find out more about Ubuntu's features and how we support developers and organisations below.
Discover Ubuntu Desktop
Check out the blog ›
Ubuntu 24.04 LTS
The latest LTS version of Ubuntu, for desktop PCs and laptops. LTS stands for long-term support — which means five years of free security and maintenance updates, extended to 10 years with Ubuntu Pro.
Download 24.04 LTS
6GB
For other versions of Ubuntu Desktop including torrents, the network installer, a list of local mirrors and past releases check out our alternative downloads.
What's new
System requirements
How to install
New Desktop installer with support for autoinstall
New App Center and Firmware Updater applications
GNOME 46 with support for quarter screen tiling
Advanced Active Directory Group Policy Object support for Ubuntu Pro users

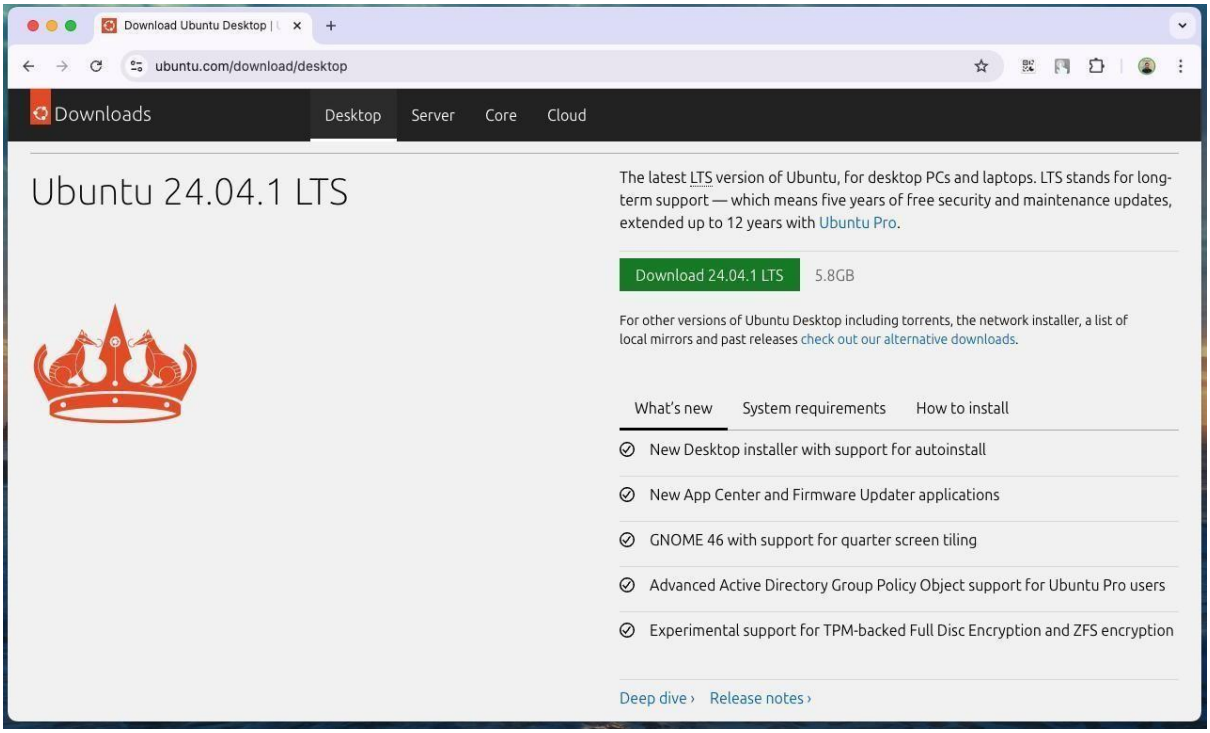
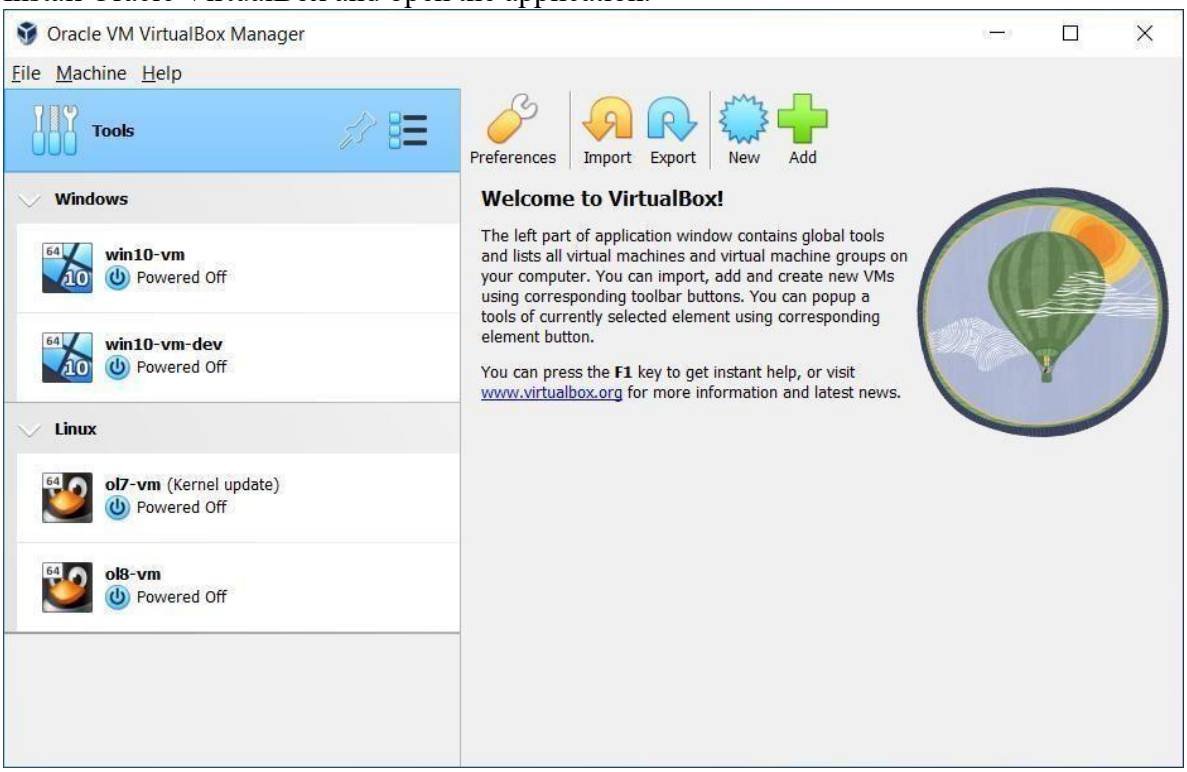


Figure 1.1 Ubuntu Download Page

Step 2: Open VirtualBox
Install Oracle VirtualBox and open the application.



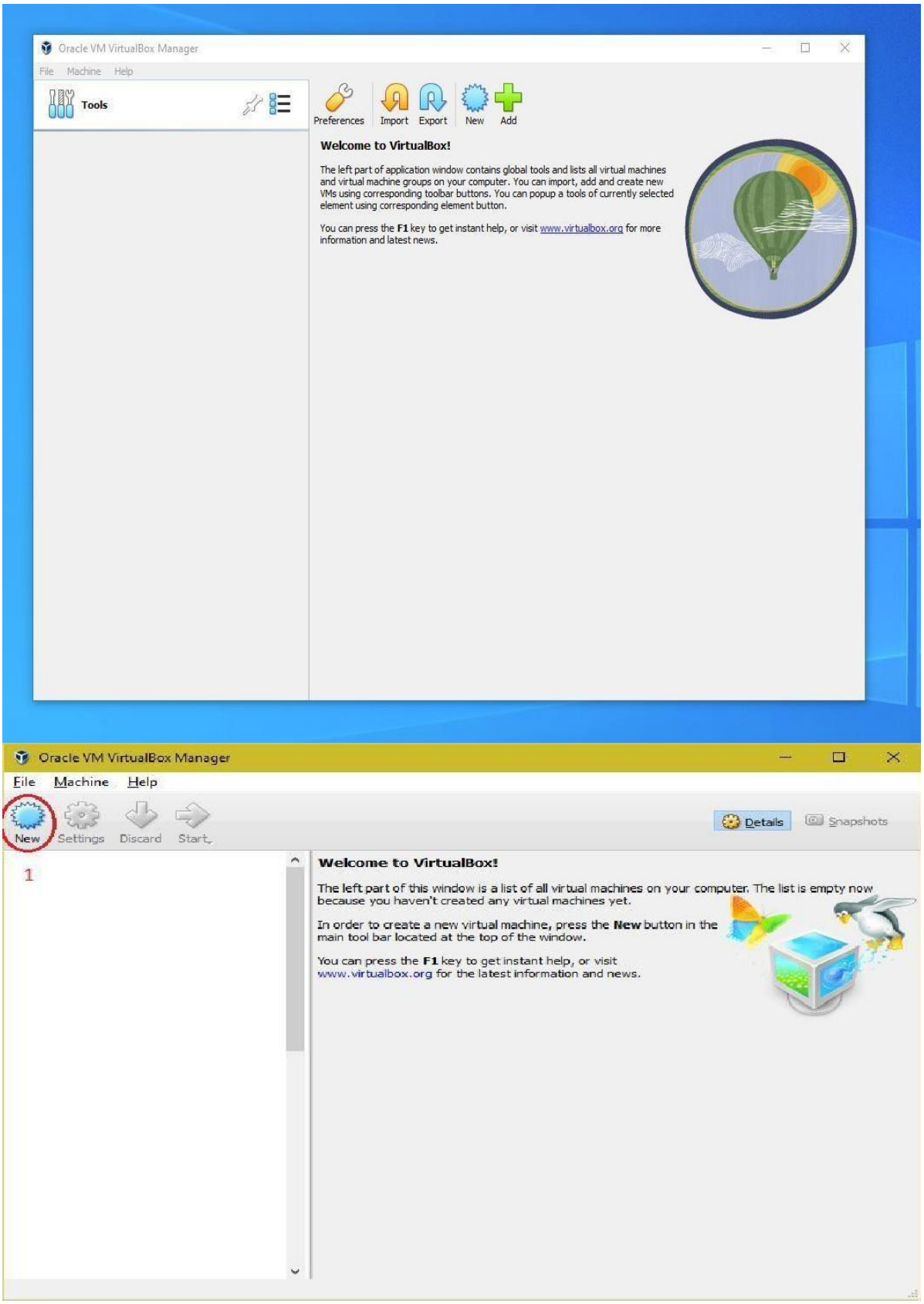
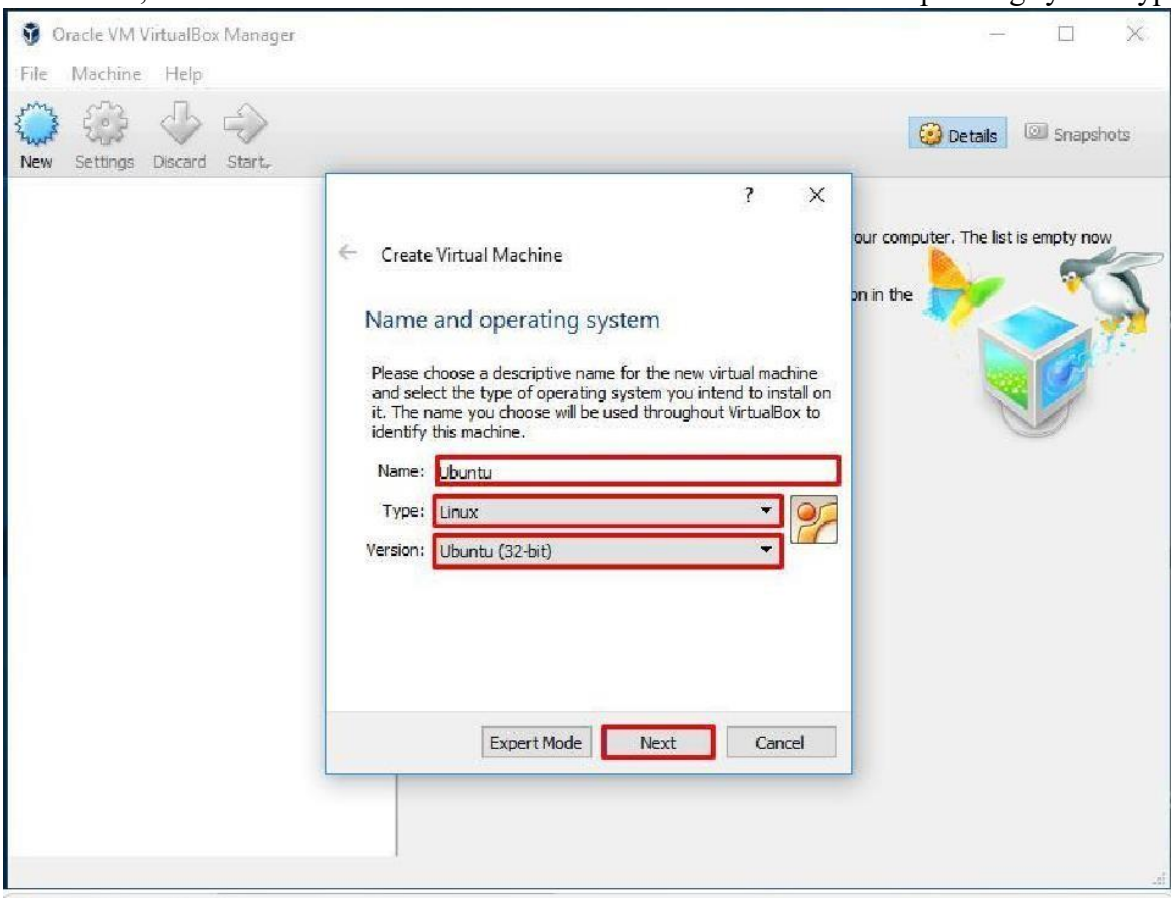
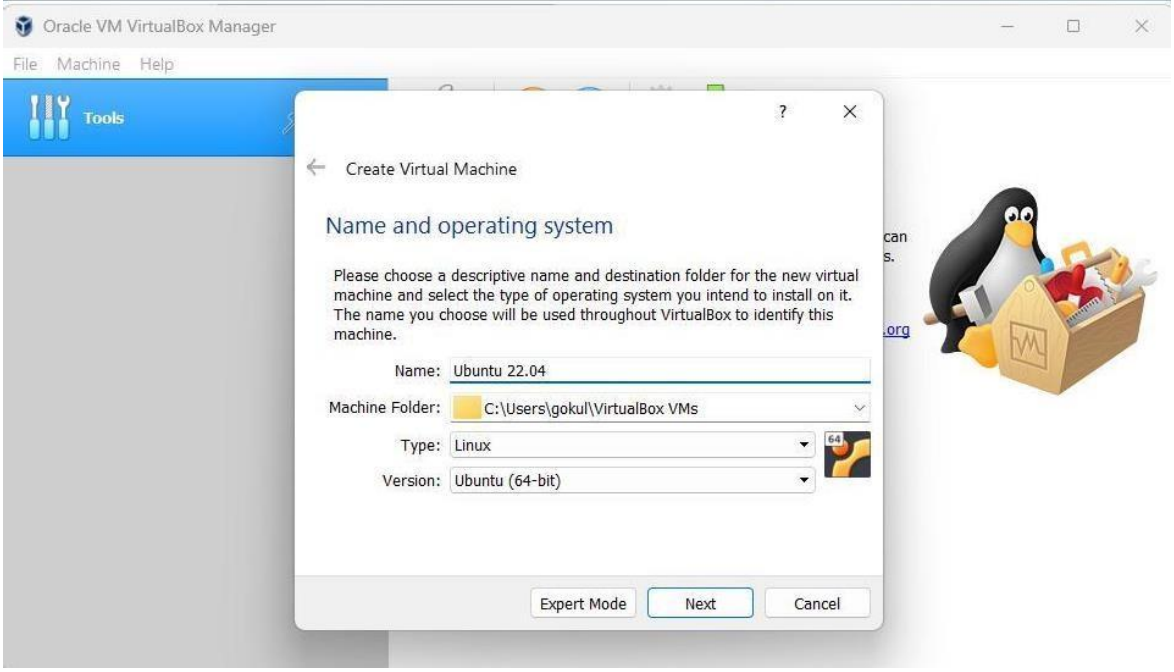


Figure 1.2 VirtualBox Main Window

Step 3: Create a New Virtual Machine
Click New, enter the machine name as Ubuntu and select Linux as the operating system type.





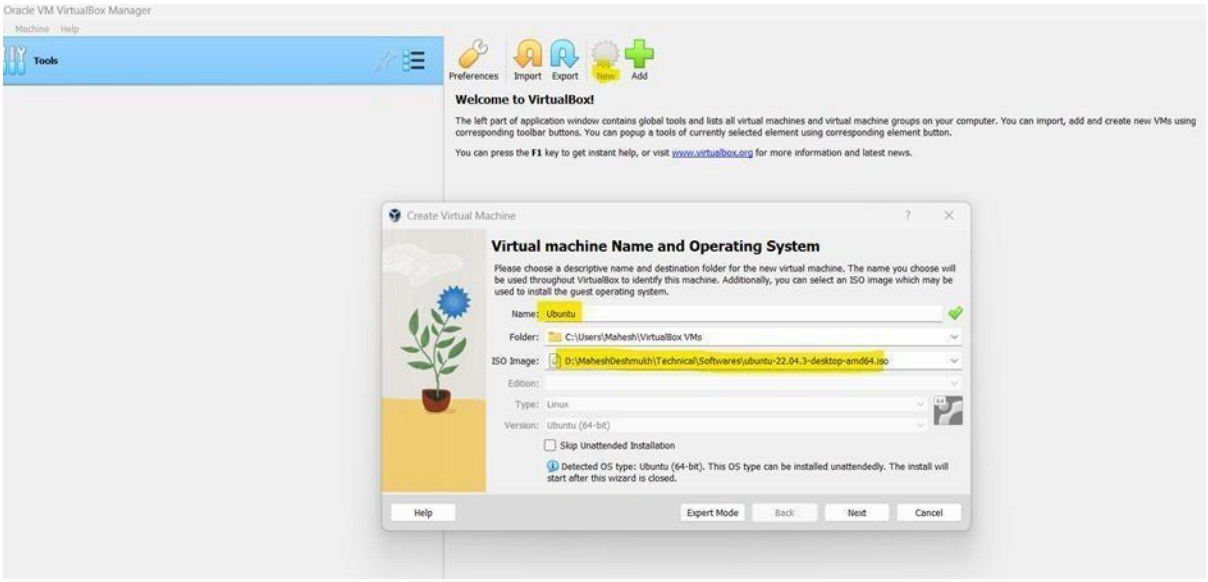
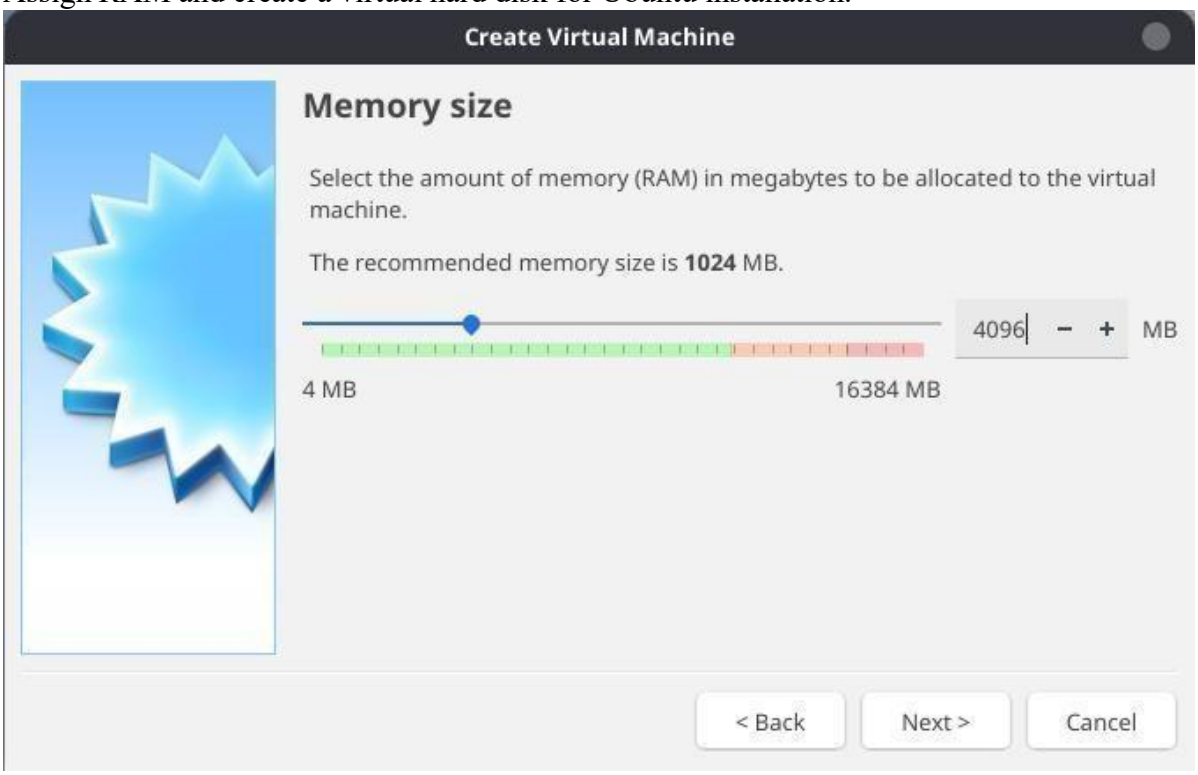


Figure 1.3 Creating a New Virtual Machine

Step 4: Allocate Memory and Storage
Assign RAM and create a virtual hard disk for Ubuntu installation.



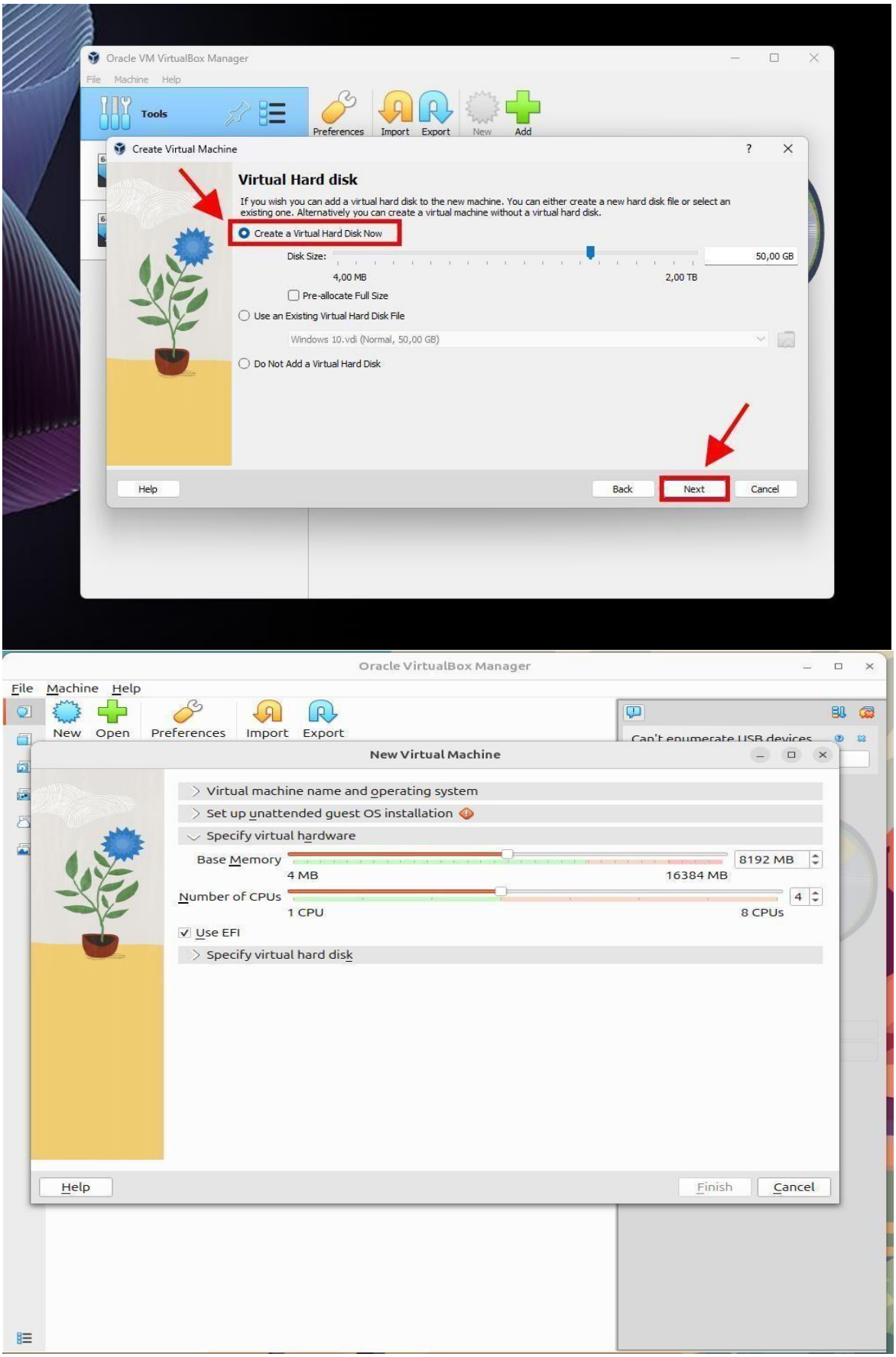


Figure 1.4 Memory and Storage Configuration

Step 5: Attach Ubuntu ISO File

Attach the downloaded Ubuntu ISO image through Storage Settings.

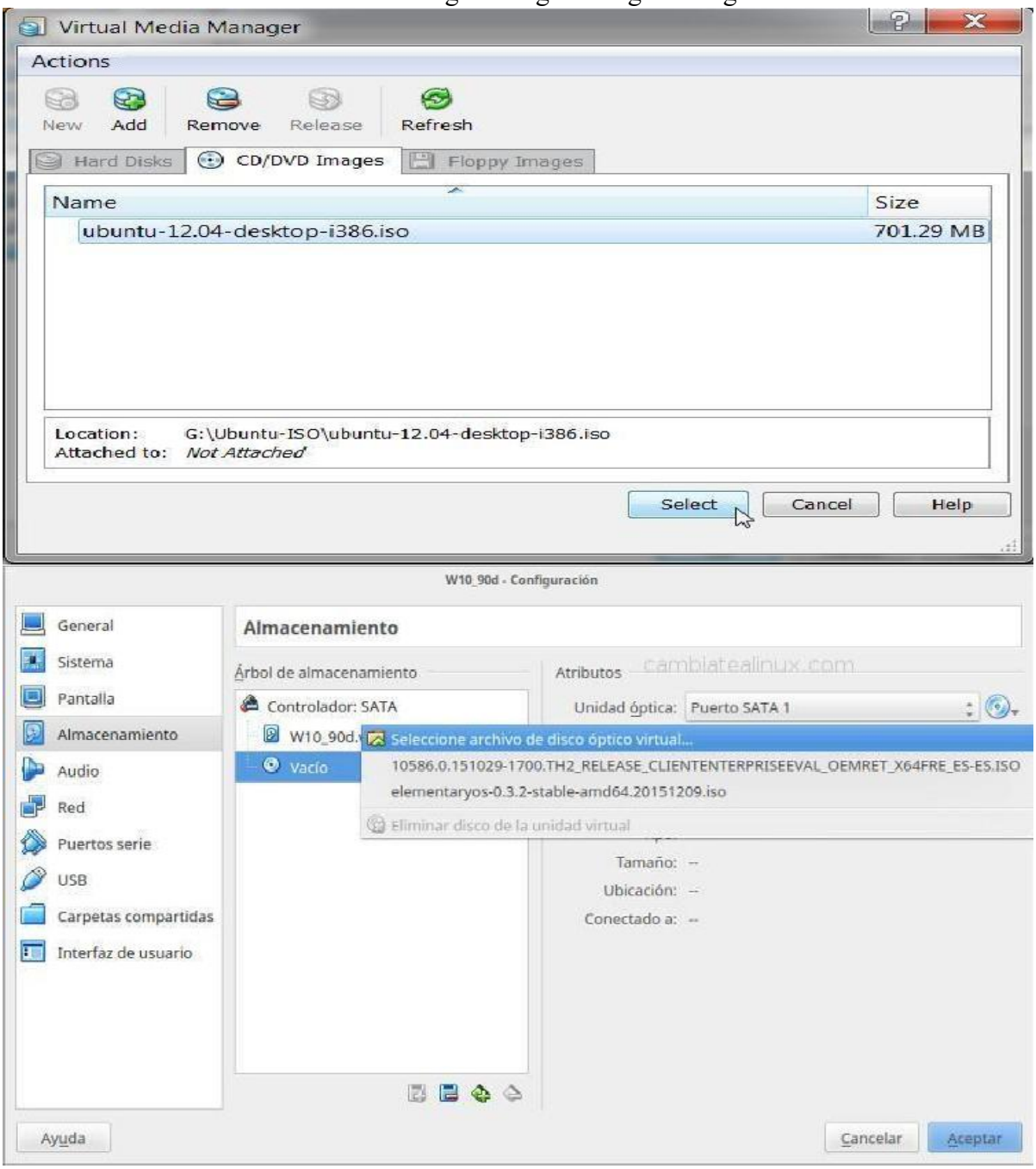
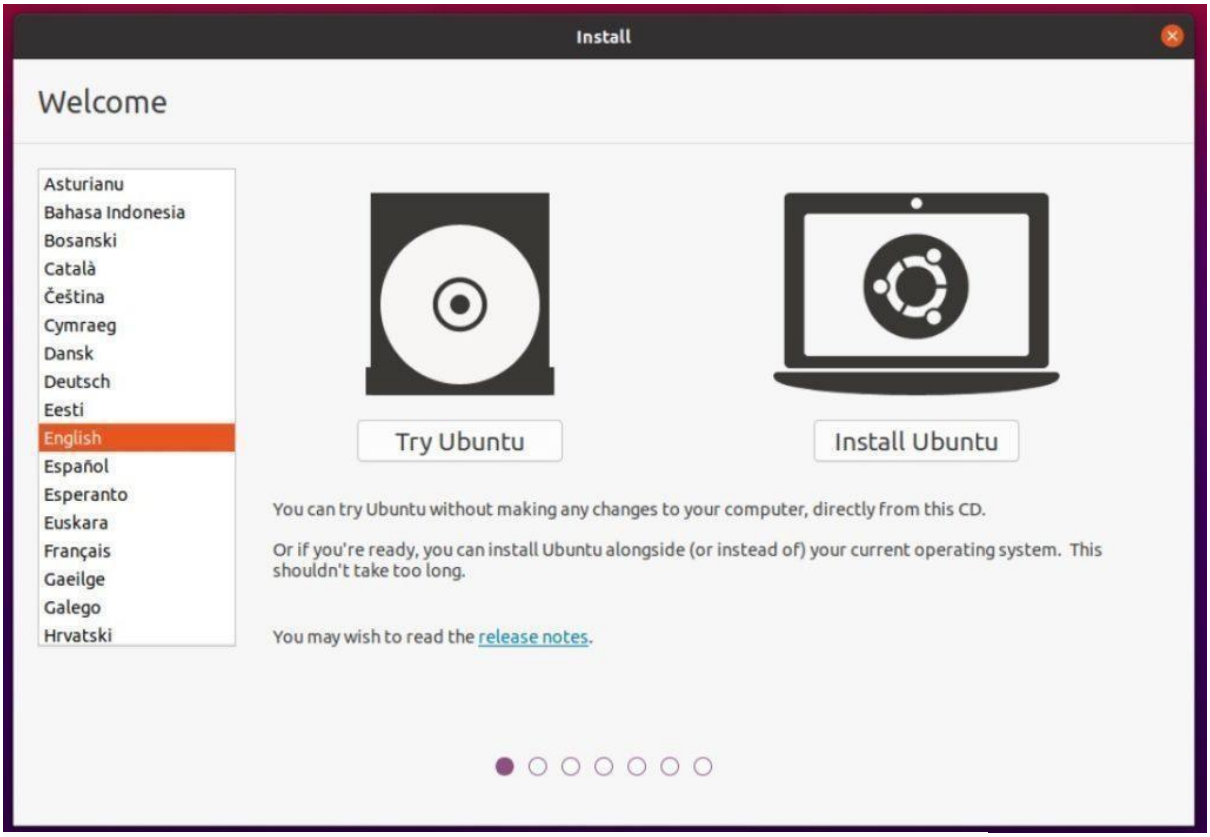
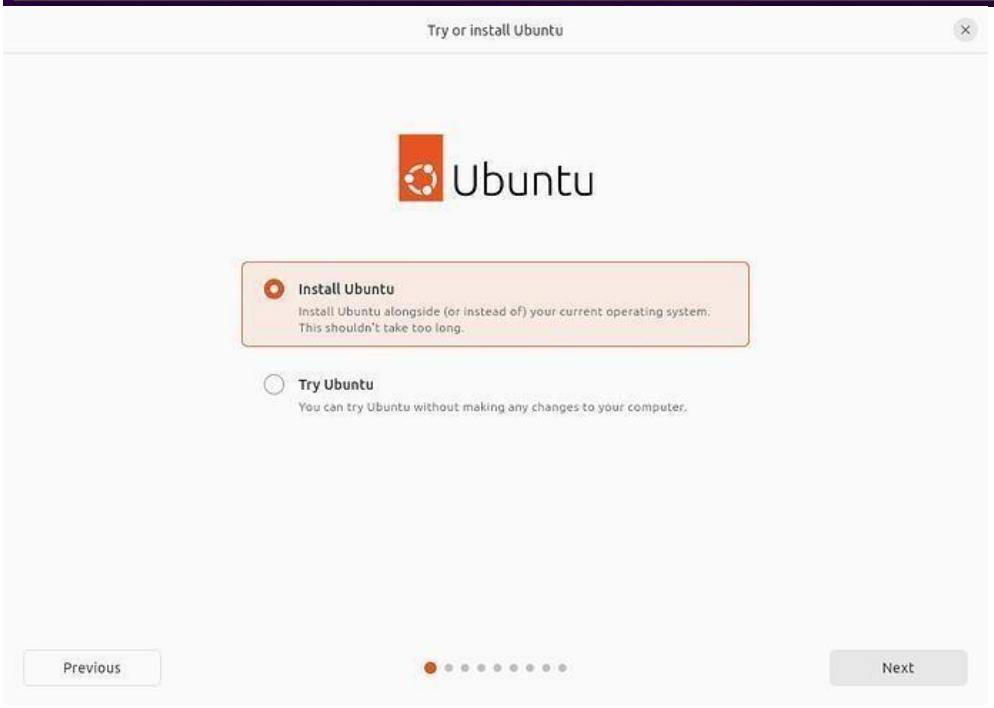


Figure 1.5 Attaching Ubuntu ISO File

Step 6: Start Ubuntu Installation
Start the virtual machine and choose Install Ubuntu.





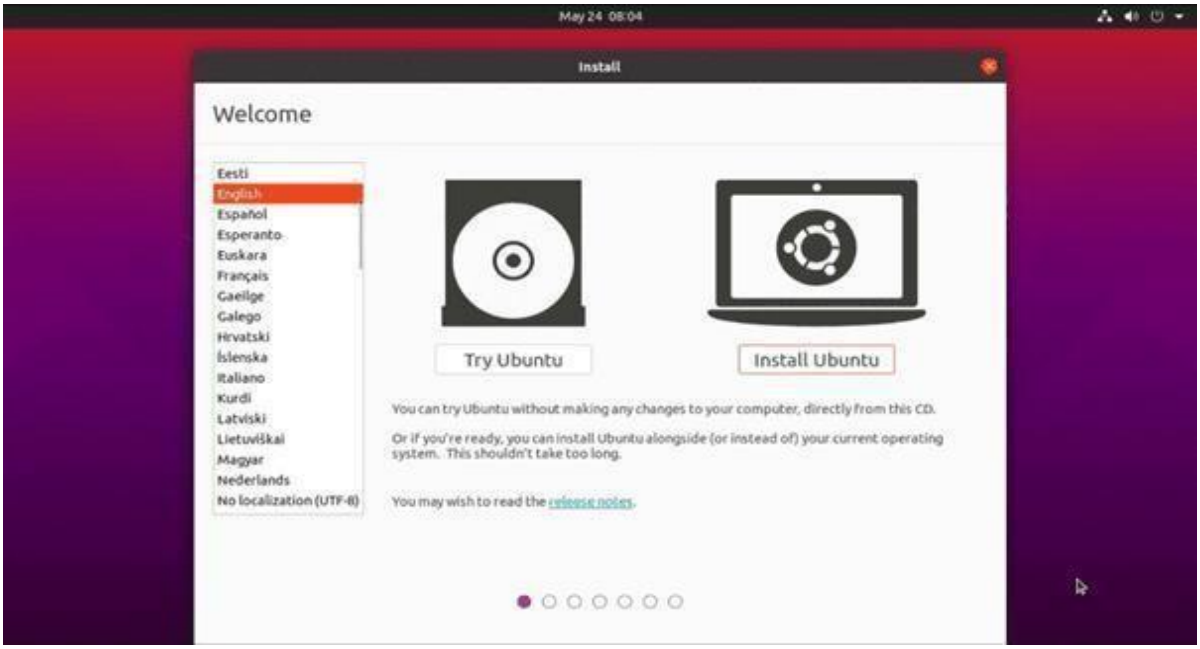
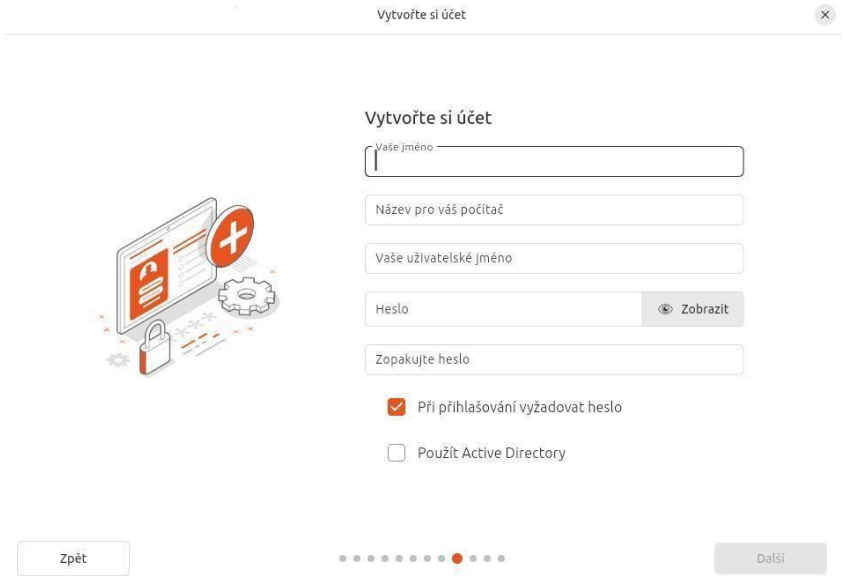


Figure 1.6 Ubuntu Installation Screen

Step 7: Configure User Information
Enter username, password and other installation details.



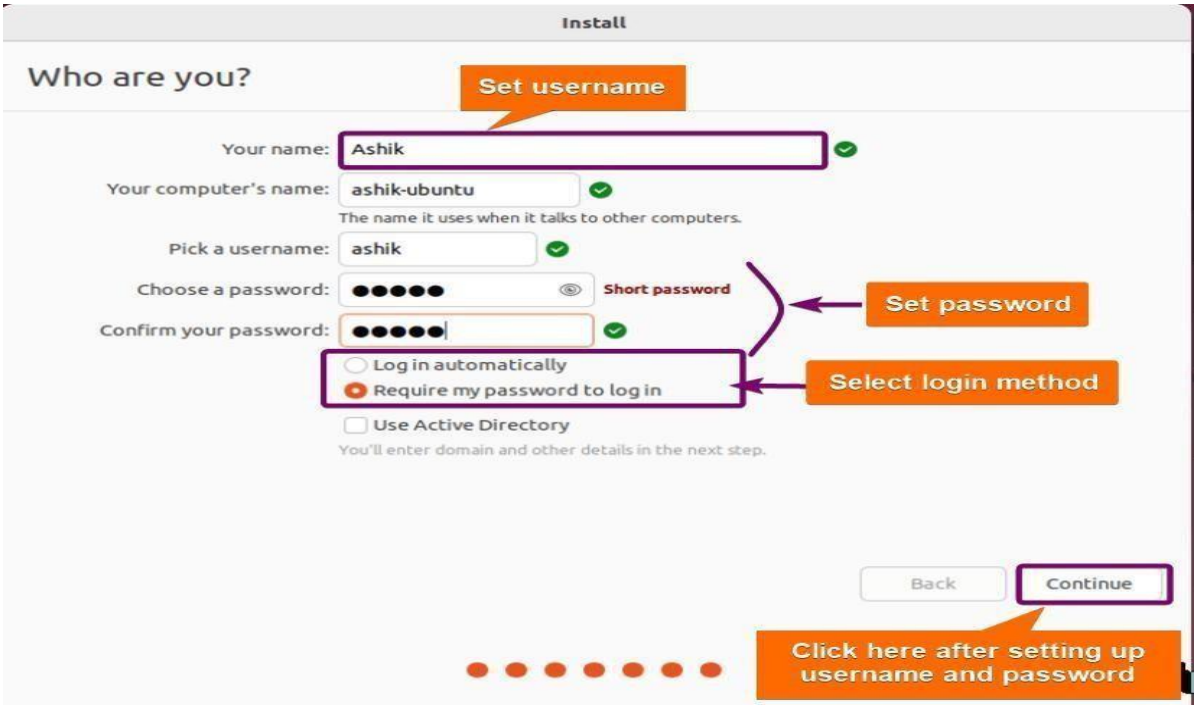
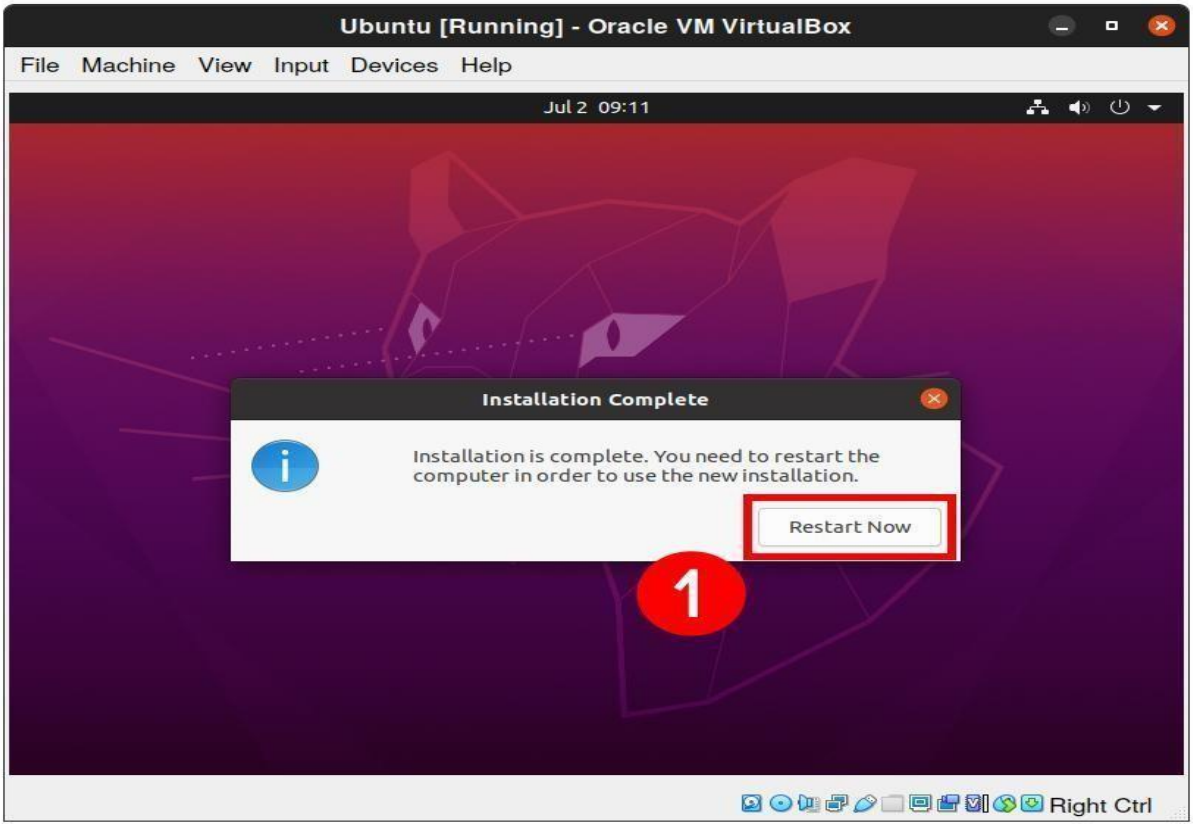


Figure 1.7 User Account Configuration

Step 8: Complete Installation
Wait for installation to complete and restart the virtual machine.





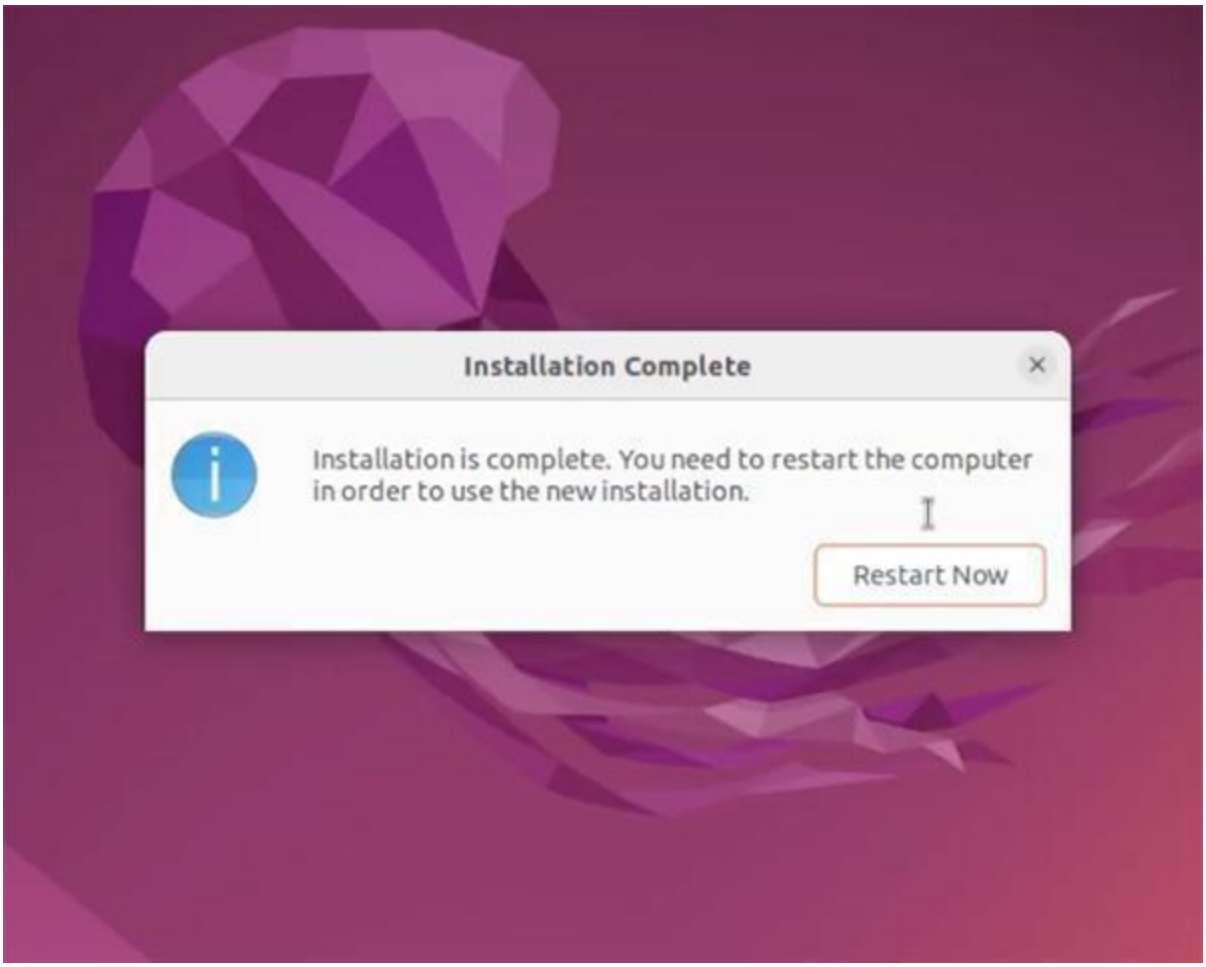
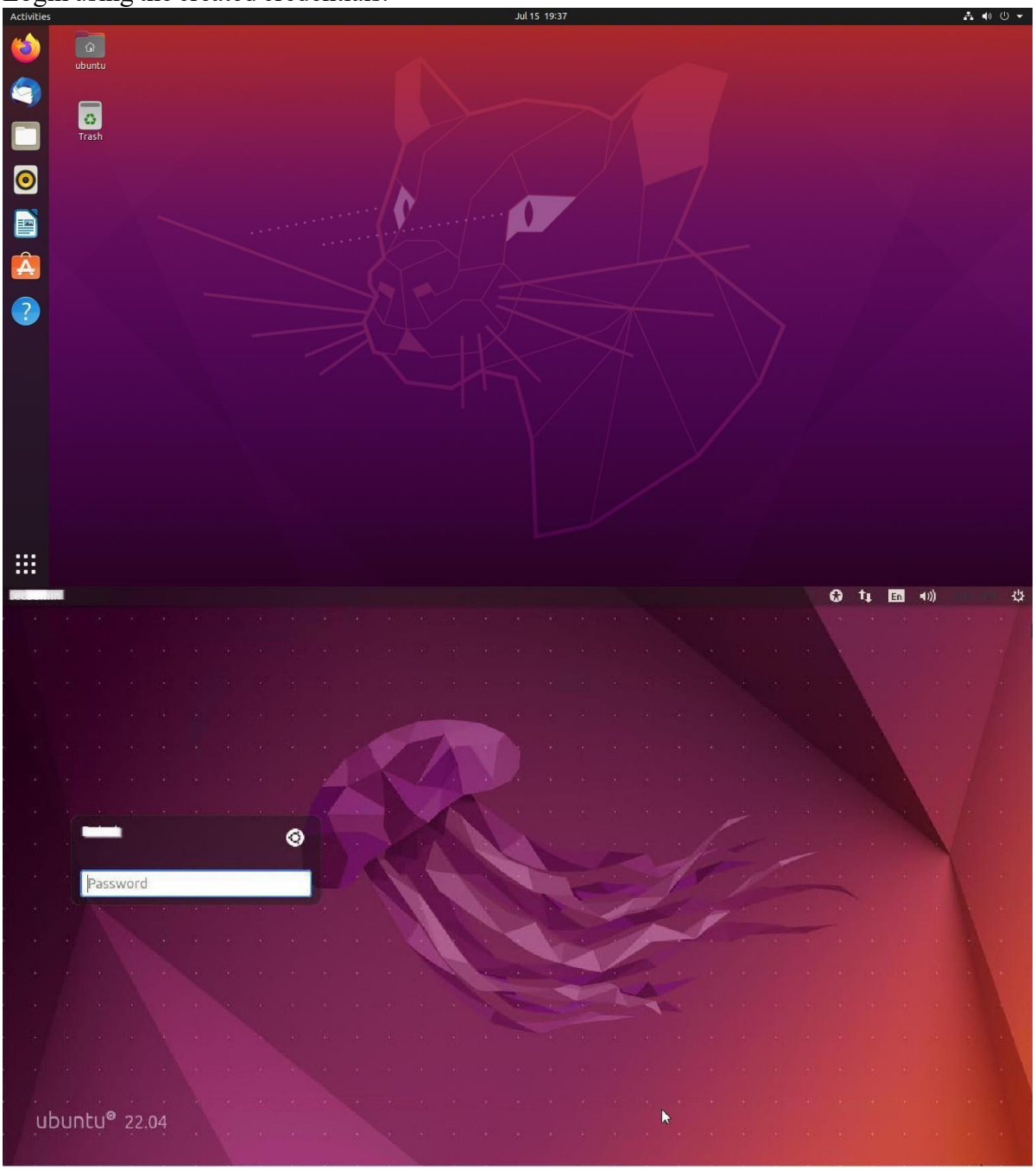


Figure 1.8 Installation Completed

Step 9: Login to Ubuntu Desktop
Login using the created credentials.



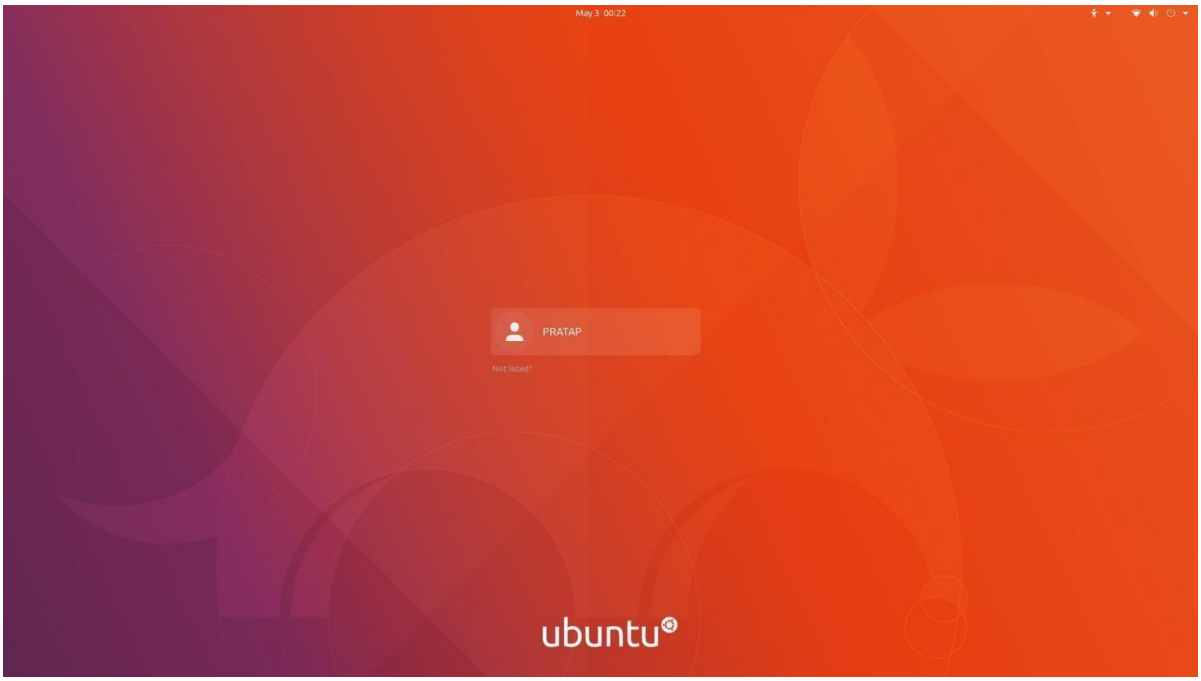
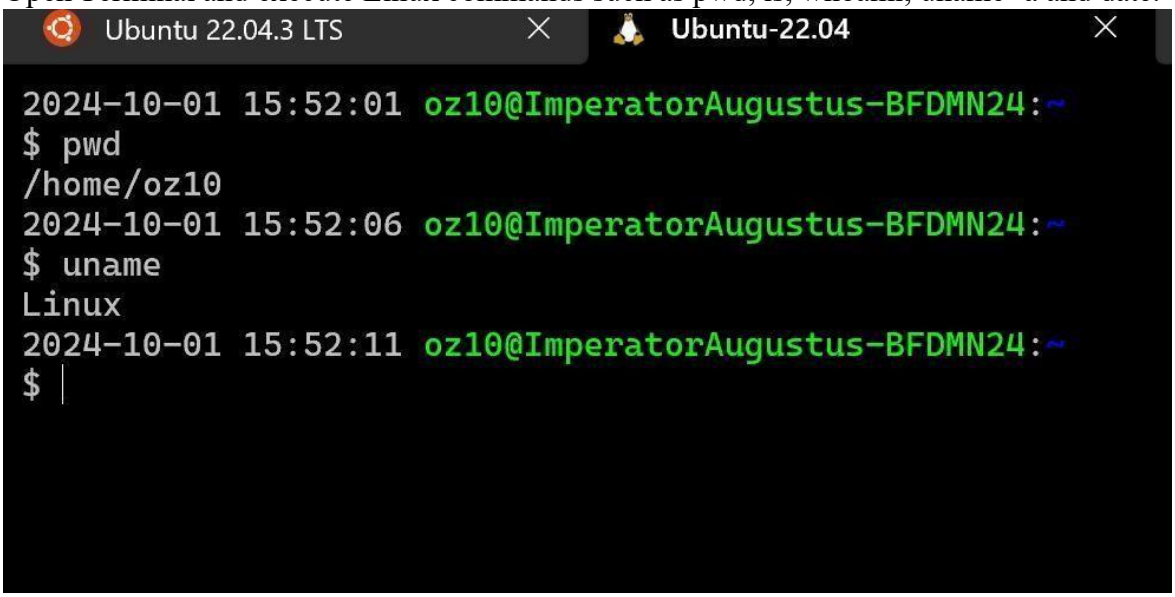


Figure 1.9 Ubuntu Desktop

Step 10: Verify Installation Using Terminal
Open Terminal and execute Linux commands such as pwd, ls, whoami, uname -a and date.



```
ifpe@ifpe-Desktop:/usr$ uname -a
Linux ifpe-Desktop 6.8.0-78-generic #78~22.04.1-Ubuntu
SMP PREEMPT_DYNAMIC Wed Aug 13 14:32:06 UTC 2 x86_64
x86_64 x86_64 GNU/Linux
ifpe@ifpe-Desktop:/usr$ ls -l
total 152
drwxr-xr-x   2 root root 61440 ago 27 08:07 bin
drwxr-xr-x   2 root root  4096 fev 28  2025 games
drwxr-xr-x  42 root root 16384 jul  7 10:09 include
drwxr-xr-x 122 root root  4096 ago 27 08:20 lib
drwxr-xr-x   2 root root  4096 set 18  2024 lib32
drwxr-xr-x   2 root root  4096 jun  5 16:52 lib64
drwxr-xr-x  22 root root 12288 mar 24 08:00 libexec
drwxr-xr-x   2 root root  4096 set 18  2024 libx32
drwxr-xr-x  10 root root  4096 set 18  2024 local
drwxr-xr-x   2 root root 20480 ago 27 08:07 sbin
drwxr-xr-x 296 root root 12288 ago  6 14:27 share
drwxr-xr-x   9 root root  4096 ago 27 08:20 src
ifpe@ifpe-Desktop:/usr$
```

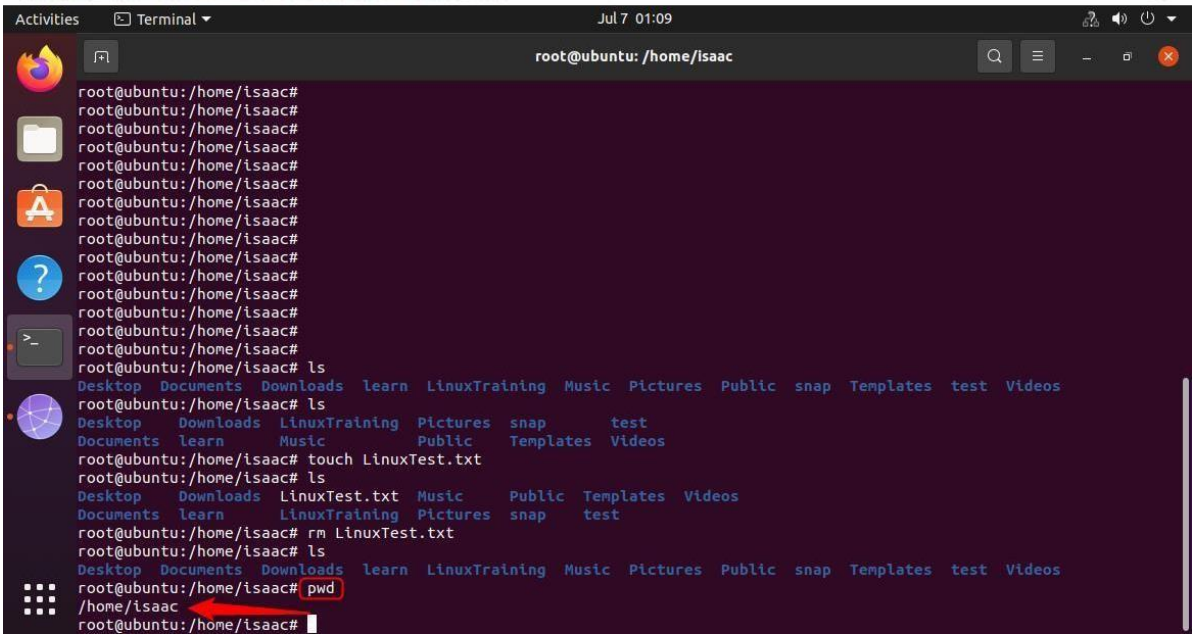


Figure 1.10 Verification of Ubuntu Installation

RESULT:

Thus, Ubuntu Linux Operating System was installed successfully and verified using basic Linux commands.

EX.NO : 2 LINUX COMMANDS AND SHELL SCRIPTING
DATE :

AIM
To study and execute basic Linux commands and shell scripting
1) Command : pwd (Print Working Directory)
Syntax:
pwd
Example:
student@ubuntu:~$ pwd
/home/student
student@ubuntu:~$

2) Command : ls (List Files and Directories)
Syntax:
ls
Example:
student@ubuntu:~$ ls
Documents Downloads Music Pictures Videos
student@ubuntu:~$

3) Command : mkdir (Make Directory)
Syntax:
mkdir directoryname
Example:
student@ubuntu:~$ mkdir programs
student@ubuntu:~$ ls
Documents Downloads programs
student@ubuntu:~$

4) Command : cd (Change Directory)
Syntax:
cd directoryname
Example:
student@ubuntu:~$ cd programs
student@ubuntu:~/programs$

5) Command : cat (Create and Display File)
Syntax:
cat > filename
Example:
student@ubuntu:~$ cat > one.txt
Welcome to Linux Lab
Operating System
Ctrl + D
student@ubuntu:~$
Display File Contents

```
student@ubuntu:~$ cat one.txt
Welcome to Linux Lab
Operating System
student@ubuntu:~$
```

6) Command : cp (Copy File)
Syntax:
cp sourcefile destinationfile
Example:

```
student@ubuntu:~$ cp one.txt copy.txt
student@ubuntu:~$
```

7) Command : mv (Move/Rename File)
Syntax:
mv sourcefile destinationfile
Example:

```
student@ubuntu:~$ mv copy.txt new.txt
student@ubuntu:~$
```

8) Command : rm (Remove File)
Syntax:
rm filename
Example:

```
student@ubuntu:~$ rm new.txt
student@ubuntu:~$
```

9) Command : date
Syntax:
date
Example:

```
student@ubuntu:~$ date
Thu Jun 25 10:30:15 IST 2026
student@ubuntu:~$
```

10) Command : cal
Syntax:
cal
Example:

```
student@ubuntu:~$ cal
     June 2026
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30
student@ubuntu:~$
```

SHELL SCRIPTING

Program

```
echo "Welcome to Operating System Lab"
echo "Linux Shell Scripting"
```

Execution

```
student@ubuntu:~$ nano hello.sh
student@ubuntu:~$ chmod +x hello.sh
student@ubuntu:~$ ./hello.sh
Welcome to Operating System Lab
Linux Shell Scripting
student@ubuntu:~$
```

RESULT:

Thus the study and execution of Linux commands and shell scripting has been completed successfully.

EX.NO : 3 **PROCESS CREATION USING fork(), exec(), wait()**

DATE :

AIM

To write a program using system calls fork(), exec(), getpid(), getppid() and wait().

ALGORITHM

Step 1 : Declare the variable pid.
Step 2 : Get the pid value using fork() system call.
Step 3 : If pid is less than zero, display "Fork Failed".
Step 4 : If pid is equal to zero, execute the child process using exec().
Step 5 : If pid is greater than zero, the parent process waits for the child process using wait().
Step 6 : Display the process IDs using getpid() and getppid().
Step 7 : Stop the program.

PROGRAM

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/wait.h>
int main()
{
   int pid;

   pid = fork();

   if(pid < 0)
   {
      printf("Fork Failed");
   }
   else if(pid == 0)
   {
      printf("Child Process ID : %d\n", getpid());
      printf("Parent Process ID : %d\n", getppid());
      execlp("cal","cal",NULL);
   }
   else
   {
      wait(NULL);
      printf("Parent Process ID : %d\n", getpid());
   }

   return 0;
}
```

OUTPUT

```
Child Process ID : 2451
Parent Process ID : 2450

    June 2026
Su Mo Tu We Th Fr Sa
...
Parent Process ID : 2450
```

RESULT:

Thus the program using system calls fork(), exec(), getpid(), getppid() and wait() was executed successfully.

EX.NO : 4 MULTI-THREADING USING POSIX THREADS
DATE :

AIM
To write a C program to implement multithreading using POSIX threads.

ALGORITHM
Step 1 : Start the program.
Step 2 : Include the required header files for POSIX threads.
Step 3 : Create a thread function to perform the required task.
Step 4 : Create multiple threads using pthread_create().
Step 5 : Execute the threads concurrently.
Step 6 : Wait for the threads to complete using pthread_join().
Step 7 : Display the output from each thread.
Step 8 : Terminate the program.

PROGRAM
```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
void *thread_fun(void *arg)
{
   int num = *(int *)arg;
   printf("Thread %d is executing\n", num);
   pthread_exit(NULL);
}
int main()
{
   pthread_t t1, t2;
   int a = 1, b = 2;
   pthread_create(&t1, NULL, thread_fun, &a);
   pthread_create(&t2, NULL, thread_fun, &b);
   pthread_join(t1, NULL);
   pthread_join(t2, NULL);
   return 0;
}
```

OUTPUT
```
Thread 1 is executing
Thread 2 is executing
```

RESULT:
Thus the program for implementing multithreading using POSIX threads was executed successfully.

EX.NO : 5.1 FIRST COME FIRST SERVE (FCFS) CPU SCHEDULING
DATE :

AIM

To write a C program to implement the First Come First Serve (FCFS) CPU Scheduling algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.
Step 2: Read the number of processes and their burst times.
Step 3: Assign the waiting time of the first process as zero.
Step 4: Calculate the waiting time for the remaining processes.
Step 5: Calculate the turnaround time for each process.
Step 6: Display the Gantt Chart.
Step 7: Calculate the average waiting time and average turnaround time.
Step 8: Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int bt[20], wt[20], tat[20];
   int n, i;
   float awt = 0, atat = 0;
   printf("Enter the Number of Processes: ");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("Enter Burst Time for Process P%d: ",i+1);
      scanf("%d",&bt[i]);
   }
   wt[0] = 0;
   for(i=1;i<n;i++)
      wt[i] = wt[i-1] + bt[i-1];
   for(i=0;i<n;i++)
   {
      tat[i] = wt[i] + bt[i];
      awt += wt[i];
      atat += tat[i];
   }
   printf("\nProcess\tBT\tWT\tTAT\n");
   for(i=0;i<n;i++)
   {
      printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);
   }
   printf("\nAverage Waiting Time = %.2f",awt/n);
   printf("\nAverage Turnaround Time = %.2f",atat/n);
   return 0;
}
```

OUTPUT

```
Enter the Number of Processes: 4
Enter Burst Time for Process P1: 5
Enter Burst Time for Process P2: 3
Enter Burst Time for Process P3: 8
Enter Burst Time for Process P4: 6

Process   BT   WT   TAT
P1        5 0    5
P2        3 5    8
P3        8 8    16
P4        6 16    22

Average Waiting Time = 7.25
Average Turnaround Time = 12.75
Gantt Chart
| P1 | P2 | P3 | P4 |
0    5    8   16   22
```

RESULT:

Thus the First Come First Serve (FCFS) CPU Scheduling algorithm was implemented successfully and the Gantt Chart was generated successfully.

EX.NO : 5.2 SHORTEST JOB FIRST (SJF) CPU SCHEDULING
DATE :

AIM

To write a C program to implement the Shortest Job First (SJF) CPU Scheduling algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.
Step 2: Read the number of processes and their burst times.
Step 3: Arrange the processes in ascending order of burst time.
Step 4: Calculate the waiting time for each process.
Step 5: Calculate the turnaround time for each process.
Step 6: Display the Gantt Chart.
Step 7: Calculate the average waiting time and average turnaround time.
Step 8: Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int bt[20], wt[20], tat[20], p[20];
   int n, i, j, temp;
   float awt = 0, atat = 0;
   printf("Enter the Number of Processes: ");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      printf("Enter Burst Time for Process P%d: ",i+1);
      scanf("%d",&bt[i]);
      p[i]=i+1;
   }
   /* Sorting according to Burst Time */
   for(i=0;i<n-1;i++)
   {
      for(j=i+1;j<n;j++)
      {
         if(bt[i]>bt[j])
         {
            temp=bt[i];
            bt[i]=bt[j];
            bt[j]=temp;

            temp=p[i];
            p[i]=p[j];
            p[j]=temp;
         }
      }
   }
```

```
    wt[0]=0;
    for(i=1;i<n;i++)
        wt[i]=wt[i-1]+bt[i-1];
    for(i=0;i<n;i++)
    {
        tat[i]=wt[i]+bt[i];
        awt+=wt[i];
        atat+=tat[i];
    }
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\n",p[i],bt[i],wt[i],tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

OUTPUT

```
Enter the Number of Processes: 4
Enter Burst Time for Process P1: 6
Enter Burst Time for Process P2: 8
Enter Burst Time for Process P3: 7
Enter Burst Time for Process P4: 3

Process   BT   WT   TAT
P4        3 0    3
P1        6 3    9
P3        7 9    16
P2        8 16    24

Average Waiting Time = 7.00
Average Turnaround Time = 13.00
Gantt Chart
| P4 | P1 | P3 | P2 |
0    3    9   16   24
```

RESULT:

Thus the Shortest Job First (SJF) CPU Scheduling algorithm was implemented successfully and the Gantt Chart was generated successfully.

EX.NO : 5.3 ROUND ROBIN (RR) CPU SCHEDULING
DATE :

AIM
To write a C program to implement the Round Robin (RR) CPU Scheduling algorithm and generate the Gantt Chart.

ALGORITHM
Step 1: Start the program.
Step 2: Read the number of processes, burst times, and time quantum.
Step 3: Initialize the remaining burst time of each process.
Step 4: Execute each process for the given time quantum.
Step 5: If the remaining burst time is greater than the time quantum, place the process back in the ready queue.
Step 6: Repeat the process until all processes are completed.
Step 7: Calculate the waiting time and turnaround time for each process.
Step 8: Display the Gantt Chart, average waiting time, and average turnaround time.
Step 9: Stop the program.

PROGRAM
```
#include<stdio.h>
int main()
{
   int bt[20], rem_bt[20], wt[20], tat[20];
   int n, tq, i, time = 0, remain;
   float awt = 0, atat = 0;
   printf("Enter the Number of Processes: ");
   scanf("%d", &n);
   remain = n;
   for(i = 0; i < n; i++)
   {
      printf("Enter Burst Time for Process P%d: ", i + 1);
      scanf("%d", &bt[i]);
      rem_bt[i] = bt[i];
   }
   printf("Enter Time Quantum: ");
   scanf("%d", &tq);
   while(remain != 0)
   {
      for(i = 0; i < n; i++)
      {
         if(rem_bt[i] > 0)
         {
            if(rem_bt[i] <= tq)
            {
               time += rem_bt[i];
               wt[i] = time - bt[i];
               rem_bt[i] = 0;
               remain--;
```

```
            }
            else
            {
                rem_bt[i] -= tq;
                time += tq;
            }
        }
    }
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
    {
        tat[i] = bt[i] + wt[i];
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", awt / n);
    printf("\nAverage Turnaround Time = %.2f", atat / n);
    return 0;
}
```

OUTPUT

```
Enter the Number of Processes: 3

Enter Burst Time for Process P1: 10
Enter Burst Time for Process P2: 5
Enter Burst Time for Process P3: 8

Enter Time Quantum: 2

Process  BT  WT  TAT
P1       10  13   23
P2        5  10   15
P3        8  13   21

Average Waiting Time = 12.00
Average Turnaround Time = 19.67
Gantt Chart
|P1|P2|P3|P1|P2|P3|P1|P2|P3|P1|P3|P1|
0  2  4  6  8 10 12 14 15 17 19 21 23
```

RESULT:

Thus the Round Robin (RR) CPU Scheduling algorithm was implemented successfully and the Gantt Chart was generated successfully.

EX.NO : 5.4 **PRIORITY CPU SCHEDULING**
DATE :

AIM

To write a C program to implement the Priority CPU Scheduling algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.
Step 2: Read the number of processes, burst times, and priority values.
Step 3: Arrange the processes in ascending order of priority (smaller number indicates higher priority).
Step 4: Calculate the waiting time for each process.
Step 5: Calculate the turnaround time for each process.
Step 6: Display the Gantt Chart.
Step 7: Calculate the average waiting time and average turnaround time.
Step 8: Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int bt[20], wt[20], tat[20], p[20], pr[20];
   int n, i, j, temp;
   float awt = 0, atat = 0;
   printf("Enter the Number of Processes: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++)
   {
      p[i] = i + 1;
      printf("Enter Burst Time for Process P%d: ", i + 1);
      scanf("%d", &bt[i]);
      printf("Enter Priority for Process P%d: ", i + 1);
      scanf("%d", &pr[i]);
   }
   /* Sorting based on Priority */
   for(i = 0; i < n - 1; i++)
   {
      for(j = i + 1; j < n; j++)
      {
         if(pr[i] > pr[j])
         {
            temp = pr[i];
            pr[i] = pr[j];
            pr[j] = temp;
            temp = bt[i];
            bt[i] = bt[j];
            bt[j] = temp;
            temp = p[i];
            p[i] = p[j];
```

```
            p[j] = temp;
          }
        }}
    wt[0] = 0;
    for(i = 1; i < n; i++)
      wt[i] = wt[i - 1] + bt[i - 1];
    for(i = 0; i < n; i++)
    {
      tat[i] = wt[i] + bt[i];
      awt += wt[i];
      atat += tat[i];
    }
    printf("\nProcess\tPriority\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
    {
      printf("P%d\t%d\t\t%d\t%d\t%d\n", p[i], pr[i], bt[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", awt / n);
    printf("\nAverage Turnaround Time = %.2f", atat / n);
    return 0;
}
```

OUTPUT

```
Enter the Number of Processes: 4
Enter Burst Time for Process P1: 5
Enter Priority for Process P1: 3
Enter Burst Time for Process P2: 3
Enter Priority for Process P2: 1
Enter Burst Time for Process P3: 8
Enter Priority for Process P3: 4
Enter Burst Time for Process P4: 6
Enter Priority for Process P4: 2

Process   Priority   BT   WT   TAT
P2    1           3 0    3
P4    2           6 3    9
P1    3           5 9    14
P3    4           8 14    22

Average Waiting Time = 6.50
Average Turnaround Time = 12.00
Gantt Chart
| P2 | P4 | P1 | P3 |
0    3    9   14   22
```

RESULT:

Thus the Priority CPU Scheduling algorithm was implemented successfully and the Gantt Chart was generated successfully.

EX.NO : 5.5 MULTILEVEL QUEUE CPU SCHEDULING
DATE :

AIM

To write a C program to implement the Multilevel Queue CPU Scheduling algorithm and generate the Gantt Chart.

ALGORITHM

Step 1: Start the program.
Step 2: Read the number of processes, burst times, and queue numbers.
Step 3: Divide the processes into different queues based on their queue numbers.
Step 4: Execute the processes in the higher priority queue first.
Step 5: Execute the processes in the lower priority queue after completing the higher priority queue.
Step 6: Calculate the waiting time and turnaround time for each process.
Step 7: Display the Gantt Chart.
Step 8: Display the average waiting time and average turnaround time.
Step 9: Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int bt[20], wt[20], tat[20], q[20], p[20];
   int n, i, j, temp;
   float awt = 0, atat = 0;
   printf("Enter the Number of Processes: ");
   scanf("%d",&n);
   for(i=0;i<n;i++)
   {
      p[i]=i+1;
      printf("Enter Burst Time for Process P%d: ",i+1);
      scanf("%d",&bt[i]);
      printf("Enter Queue Number (1-High, 2-Low) for P%d: ",i+1);
      scanf("%d",&q[i]);
   }
   /* Arrange processes according to Queue Priority */
   for(i=0;i<n-1;i++)
   {
      for(j=i+1;j<n;j++)
      {
         if(q[i]>q[j])
         {
            temp=q[i];
            q[i]=q[j];
            q[j]=temp;
            temp=bt[i];
            bt[i]=bt[j];
            bt[j]=temp;
```

```
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }
    wt[0]=0;
    for(i=1;i<n;i++)
        wt[i]=wt[i-1]+bt[i-1];
    for(i=0;i<n;i++)
    {
        tat[i]=wt[i]+bt[i];
        awt+=wt[i];
        atat+=tat[i];
    }
    printf("\nProcess\tQueue\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\n",p[i],q[i],bt[i],wt[i],tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

OUTPUT

```
Enter the Number of Processes: 4
Enter Burst Time for Process P1: 5
Enter Queue Number (1-High, 2-Low) for P1: 2
Enter Burst Time for Process P2: 3
Enter Queue Number (1-High, 2-Low) for P2: 1
Enter Burst Time for Process P3: 8
Enter Queue Number (1-High, 2-Low) for P3: 2
Enter Burst Time for Process P4: 6
Enter Queue Number (1-High, 2-Low) for P4: 1

Process  Queue  BT  WT  TAT
P2     1        3   0     3
P4     1        6   3     9
P1     2        5   9     14
P3     2        8   14     22

Average Waiting Time = 6.50
Average Turnaround Time = 12.00
Gantt Chart
| P2 | P4 | P1 | P3 |
0    3    9   14   22
```

RESULT:

Thus the Multilevel Queue CPU Scheduling algorithm was implemented successfully and the Gantt Chart was generated successfully.

EX.NO : 6.1 PRODUCER – CONSUMER PROBLEM
DATE :

AIM

To write a C program to implement the Producer-Consumer problem using semaphores.

ALGORITHM

Step 1: Start the program.
Step 2: Initialize the buffer, mutex, full, and empty semaphores.
Step 3: Read the number of items to be produced.
Step 4: The producer inserts an item into the buffer if it is not full.
Step 5: The consumer removes an item from the buffer if it is not empty.
Step 6: Repeat the producer and consumer operations until all items are processed.
Step 7: Display the producer and consumer operations.
Step 8: Stop the program.

PROGRAM

```
#include<stdio.h>
int mutex = 1, full = 0, empty = 3, x = 0;
int wait(int s)
{
   return (--s);
}
int signal(int s)
{
   return (++s);
}
int main()
{
   int ch;
   while(1)
   {
      printf("\n1. Producer");
      printf("\n2. Consumer");
      printf("\n3. Exit");
      printf("\nEnter your choice: ");
      scanf("%d",&ch);
      switch(ch)
      {
         case 1:
            if((mutex==1)&&(empty!=0))
            {
               mutex=wait(mutex);
               full=signal(full);
               empty=wait(empty);
               x++;
               printf("\nProducer produces item %d",x);
               mutex=signal(mutex);
            }
```

```
            else
            {
               printf("\nBuffer is Full");
            }
            break;
         case 2:
            if((mutex==1)&&(full!=0))
            {
               mutex=wait(mutex);
               full=wait(full);
               empty=signal(empty);
               printf("\nConsumer consumes item %d",x);
               x--;
               mutex=signal(mutex);
            }
            else
            {
               printf("\nBuffer is Empty");
            }
            break;
         case 3:
            return 0;
      }
   }
}
```

OUTPUT

```
1. Producer
2. Consumer
3. Exit
Enter your choice: 1
Producer produces item 1
Enter your choice: 1
Producer produces item 2
Enter your choice: 2
Consumer consumes item 2
Enter your choice: 3
```

RESULT:

Thus the Producer–Consumer problem using semaphores was implemented successfully.

EX.NO : 6.2 READERS – WRITERS PROBLEM
DATE :

AIM
To write a C program to implement the Readers-Writers problem using semaphores.

ALGORITHM
Step 1: Start the program.
Step 2: Initialize the semaphore and read count variables.
Step 3: Allow multiple readers to access the shared resource simultaneously.
Step 4: Allow only one writer to access the shared resource at a time.
Step 5: Prevent readers from accessing the resource while a writer is writing.
Step 6: Display the reader and writer operations.
Step 7: Stop the program.

PROGRAM

```
#include<stdio.h>
int mutex = 1, wrt = 1, readcount = 0;
int wait(int s)
{
   return (--s);
}
int signal(int s)
{
   return (++s);
}
int main()
{
   int ch;
   while(1)
   {
      printf("\n1. Reader");
      printf("\n2. Writer");
      printf("\n3. Exit");
      printf("\nEnter your choice: ");
      scanf("%d",&ch);
      switch(ch)
      {
         case 1:
            mutex = wait(mutex);
            readcount++;
            if(readcount == 1)
               wrt = wait(wrt);
            mutex = signal(mutex);
            printf("\nReader is Reading...");
            mutex = wait(mutex);
            readcount--;
            if(readcount == 0)
               wrt = signal(wrt);
```

```
            mutex = signal(mutex);
            break;
        case 2:
            wrt = wait(wrt);
            printf("\nWriter is Writing...");
            wrt = signal(wrt);
            break;
        case 3:
            return 0;
    }
  }
}
```

OUTPUT

```
1. Reader
2. Writer
3. Exit

Enter your choice: 1
Reader is Reading...
Enter your choice: 2
Writer is Writing...
Enter your choice: 3
```

RESULT:

Thus the Readers–Writers problem using semaphores was implemented successfully.

EX.NO : 7 **DEADLOCK AVOIDANCE USING BANKER'S ALGORITHM**
DATE :

AIM
To write a C program to implement Deadlock Avoidance using Banker's Algorithm.

ALGORITHM
Step 1: Start the program.
Step 2: Read the number of processes and resources.
Step 3: Read the Allocation matrix and Maximum matrix.
Step 4: Calculate the Need matrix using the formula:
Need = Maximum − Allocation
Step 5: Read the Available resources.
Step 6: Find a process whose Need is less than or equal to the Available resources.
Step 7: Execute the process, release its allocated resources, and update the Available resources.
Step 8: Repeat Steps 6 and 7 until all processes are executed or no safe sequence exists.
Step 9: Display the Safe Sequence if the system is in a safe state; otherwise, display that the system is in an unsafe state.
Step 10: Stop the program.

PROGRAM
```
#include<stdio.h>
int main()
{
   int alloc[10][10], max[10][10], need[10][10];
   int avail[10], finish[10]={0};
   int safe[10];
   int n, m, i, j, k=0, count=0;
   printf("Enter the number of Processes: ");
   scanf("%d",&n);
   printf("Enter the number of Resources: ");
   scanf("%d",&m);
   printf("Enter Allocation Matrix:\n");
   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         scanf("%d",&alloc[i][j]);
   printf("Enter Maximum Matrix:\n");
   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         scanf("%d",&max[i][j]);
   printf("Enter Available Resources:\n");
   for(i=0;i<m;i++)
      scanf("%d",&avail[i]);
   for(i=0;i<n;i++)
      for(j=0;j<m;j++)
         need[i][j]=max[i][j]-alloc[i][j];
   while(count<n)
   {
      int found=0;
```

```
        for(i=0;i<n;i++)
        {
           if(finish[i]==0)
           {
              for(j=0;j<m;j++)
                 if(need[i][j]>avail[j])
                    break;
              if(j==m)
              {
                 for(j=0;j<m;j++)
                    avail[j]+=alloc[i][j];
                 safe[k++]=i;
                 finish[i]=1;
                 found=1;
                 count++;
              }
           }
        }
        if(found==0)
           break;
     }
     if(count==n)
     {
        printf("\nSystem is in Safe State\n");
        printf("Safe Sequence: ");
        for(i=0;i<n;i++)
           printf("P%d ",safe[i]);
     }
     else
     {
        printf("\nSystem is in Unsafe State");
     }
     return 0;
  }
```

OUTPUT

```
Enter the number of Processes: 5
Enter the number of Resources: 3

System is in Safe State

Safe Sequence:
P1 P3 P4 P0 P2
```

RESULT:

Thus the Deadlock Avoidance using Banker's Algorithm was implemented successfully.

EX.NO : 8.1 **FIFO PAGE REPLACEMENT ALGORITHM (FIFO)**
DATE :

AIM

To write a C program to implement the FIFO Page Replacement algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of pages and page reference string.
Step 3 : Read the number of frames.
Step 4 : Check whether the page is already present in the frame.
Step 5 : If the page is not present, replace the oldest page using FIFO.
Step 6 : Count the page faults.
Step 7 : Display the total number of page faults.
Step 8 : Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int pages[20], frame[10];
   int n, f, i, j;
   int front = 0, fault = 0, found;
   printf("Enter number of pages: ");
   scanf("%d",&n);
   printf("Enter page reference string:\n");
   for(i=0;i<n;i++)
      scanf("%d",&pages[i]);
   printf("Enter number of frames: ");
   scanf("%d",&f);
   for(i=0;i<f;i++)
      frame[i]=-1;
   for(i=0;i<n;i++)
   {
      found=0;
      for(j=0;j<f;j++)
      {
         if(frame[j]==pages[i])
         {
            found=1;
            break;
         }
      }
      if(found==0)
      {
         frame[front]=pages[i];
         front=(front+1)%f;
         fault++;
      }
```

```
    }
    printf("\nTotal Page Faults = %d",fault);
    return 0;
}
```

OUTPUT

```
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3
Total Page Faults = 10
```

RESULT:

Thus the FIFO Page Replacement algorithm was implemented successfully.

EX.NO : 8.2 LEAST RECENTLY USED (LRU) PAGE REPLACEMENT ALGORITHM
DATE :

AIM

To write a C program to implement the Least Recently Used (LRU) Page Replacement algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of pages and page reference string.
Step 3 : Read the number of page frames.
Step 4 : Check whether the page is already present in the frame.
Step 5 : If the page is not present, replace the least recently used page.
Step 6 : Count the total number of page faults.
Step 7 : Display the total number of page faults.
Step 8 : Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int pages[20], frame[10], time[10];
   int n, f, i, j;
   int fault = 0, found, least;
   printf("Enter number of pages: ");
   scanf("%d",&n);
   printf("Enter page reference string:\n");
   for(i=0;i<n;i++)
      scanf("%d",&pages[i]);
   printf("Enter number of frames: ");
   scanf("%d",&f);
   for(i=0;i<f;i++)
   {
      frame[i] = -1;
      time[i] = -1;
   }
   for(i=0;i<n;i++)
   {
      found = 0;
      for(j=0;j<f;j++)
      {
         if(frame[j] == pages[i])
         {
            found = 1;
            time[j] = i;
            break;
         }
      }
```

```
        if(found == 0)
        {
            least = 0;
            for(j=1;j<f;j++){
                if(time[j] < time[least])
                    least = j;
            }
            frame[least] = pages[i];
            time[least] = i;
            fault++;
        }
    }
    printf("\nTotal Page Faults = %d", fault);
    return 0;
}
```

OUTPUT

```
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3
Total Page Faults = 9
```

RESULT:

Thus the Least Recently Used (LRU) Page Replacement algorithm was implemented successfully.

EX.NO : 8.3 OPTIMAL PAGE REPLACEMENT ALGORITHM

DATE :

AIM

To write a C program to implement the Optimal Page Replacement algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of pages and page reference string.
Step 3 : Read the number of page frames.
Step 4 : Check whether the page is already present in the frame.
Step 5 : If the page is not present, replace the page that will not be used for the longest period in the future.
Step 6 : Count the total number of page faults.
Step 7 : Display the total number of page faults.
Step 8 : Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   int pages[20], frame[10];
   int n, f, i, j, k;
   int fault = 0, found;
   int farthest, index, pos;
   printf("Enter number of pages: ");
   scanf("%d",&n);
   printf("Enter page reference string:\n");
   for(i=0;i<n;i++)
      scanf("%d",&pages[i]);
   printf("Enter number of frames: ");
   scanf("%d",&f);
    for(i=0;i<f;i++)
      frame[i] = -1;
   for(i=0;i<n;i++)
   {
      found = 0;
      for(j=0;j<f;j++)
      {
         if(frame[j] == pages[i])
         {
            found = 1;
            break;
         }
      }
      if(found == 0)
      {
         fault++;
         for(j=0;j<f;j++)
```

```
        {
            if(frame[j] == -1)
            {
                frame[j] = pages[i];
                found = 1;
                break;
            }
        }
        if(found == 0)
        {
            farthest = -1;
            index = -1;
            for(j=0;j<f;j++)
            {
                for(k=i+1;k<n;k++)
                {
                    if(frame[j] == pages[k])
                        break;
                }
                if(k == n)
                {
                    index = j;
                    break;
                }
                if(k > farthest)
                {
                    farthest = k;
                    index = j;
                }
            }
            frame[index] = pages[i];
        }
    }
  }
  printf("\nTotal Page Faults = %d", fault);
  return 0;
}
```

OUTPUT

```
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3
Total Page Faults = 7
```

RESULT:

Thus the Optimal Page Replacement algorithm was implemented successfully.

EX.NO : 9 DYNAMIC MEMORY ALLOCATION AND DEALLOCATION

DATE :

AIM

To write a C program to implement dynamic memory allocation and deallocation using malloc() and free().

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of elements.
Step 3 : Allocate memory dynamically using malloc().
Step 4 : Check whether the memory allocation is successful.
Step 5 : Read the elements and store them in the allocated memory.
Step 6 : Display the stored elements.
Step 7 : Deallocate the allocated memory using free().
Step 8 : Stop the program.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int *ptr, n, i;
   printf("Enter the number of elements: ");
   scanf("%d",&n);
   ptr = (int *)malloc(n * sizeof(int));
   if(ptr == NULL)
   {
      printf("Memory Allocation Failed");
      return 0;
   }
   printf("Enter %d elements:\n", n);
   for(i = 0; i < n; i++)
      scanf("%d", &ptr[i]);
   printf("\nThe entered elements are:\n");
   for(i = 0; i < n; i++)
      printf("%d ", ptr[i]);
   free(ptr);
   printf("\n\nMemory Deallocated Successfully.");
   return 0;
}
```

OUTPUT

```
Enter the number of elements: 5
Enter 5 elements:
10
20
30
40
50

The entered elements are:
10 20 30 40 50

Memory Deallocated Successfully.
```

RESULT:

Thus the program for dynamic memory allocation and deallocation using malloc() and free() was implemented successfully.

EX.NO : 10 FILE OPERATIONS AND FILE BUFFERING

DATE :

AIM

To write a C program to perform file operations and demonstrate file buffering.

ALGORITHM

Step 1 : Start the program.
Step 2 : Create a file using fopen() in write mode.
Step 3 : Write data into the file using fprintf().
Step 4 : Close the file using fclose().
Step 5 : Open the same file in read mode.
Step 6 : Read the contents of the file using fgets().
Step 7 : Display the contents on the screen.
Step 8 : Close the file.
Step 9 : Stop the program.

PROGRAM

```
#include<stdio.h>
int main()
{
   FILE *fp;
   char str[100];
   fp = fopen("sample.txt", "w");
   if(fp == NULL)
   {
      printf("File cannot be created.");
      return 0;
   }
   fprintf(fp, "Welcome to Operating System Lab\n");
   fprintf(fp, "File Operations and Buffering");
   fclose(fp);
   fp = fopen("sample.txt", "r");
   printf("Contents of the File:\n\n");
   while(fgets(str, sizeof(str), fp) != NULL)
   {
      printf("%s", str);
   }
   fclose(fp);
   return 0;
}
```

OUTPUT

Contents of the File:

Welcome to Operating System Lab
File Operations and Buffering

RESULT:

Thus the program for file operations and file buffering was implemented successfully.

EX.NO : 11.1 DISK SCHEDULING ALGORITHMS (FCFS)

DATE :

AIM

To write a C program to implement the First Come First Serve (FCFS) Disk Scheduling algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of disk requests.
Step 3 : Read the disk request queue.
Step 4 : Read the initial head position.
Step 5 : Calculate the seek time by finding the absolute difference between the current head position and the requested track.
Step 6 : Update the head position after servicing each request.
Step 7 : Calculate the total seek time.
Step 8 : Display the total seek time.
Step 9 : Stop the program.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int req[20], n, head, i;
   int totalSeek = 0;
   printf("Enter the number of disk requests: ");
   scanf("%d", &n);
   printf("Enter the disk request sequence:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &req[i]);
   printf("Enter the initial head position: ");
   scanf("%d", &head);
   for(i = 0; i < n; i++)
   {
      totalSeek += abs(req[i] - head);
      head = req[i];
   }
   printf("\nTotal Seek Time = %d", totalSeek);
   return 0;
}
```

OUTPUT

```
Enter the number of disk requests: 8
Enter the disk request sequence:
98
183
37
122
14
124
65
67

Enter the initial head position: 53

Total Seek Time = 640
```

RESULT:

Thus the FCFS Disk Scheduling algorithm was implemented successfully.

EX.NO : 11.2 SHORTEST SEEK TIME FIRST (SSTF) DISK SCHEDULING

DATE :

AIM

To write a C program to implement the SSTF Disk Scheduling algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of disk requests.
Step 3 : Read the disk request sequence.
Step 4 : Read the initial head position.
Step 5 : Find the nearest request from the current head position.
Step 6 : Service the request and update the head position.
Step 7 : Repeat until all requests are serviced.
Step 8 : Display the total seek time.
Step 9 : Stop the program.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int req[20], visited[20]={0};
   int n, head, i, j, min, pos;
   int seek=0;
   printf("Enter number of requests: ");
   scanf("%d",&n);
   printf("Enter requests:\n");
   for(i=0;i<n;i++)
      scanf("%d",&req[i]);
   printf("Enter initial head position: ");
   scanf("%d",&head);
   for(i=0;i<n;i++)
   {
      min=1000;
      for(j=0;j<n;j++)
      {
         if(!visited[j] && abs(head-req[j])<min)
         {
            min=abs(head-req[j]);
            pos=j;
         }
      }
      seek+=min;
      head=req[pos];
      visited[pos]=1;
```

```
    }
    printf("\nTotal Seek Time = %d",seek);
    return 0;
}OUTPUT
```

Enter number of requests: 8

Enter requests:
98 183 37 122 14 124 65 67

Enter initial head position: 53

Total Seek Time = 236

RESULT:

Thus the SSTF Disk Scheduling algorithm was implemented successfully.

EX.NO : 11.3 SCAN DISK SCHEDULING ALGORITHM

DATE :

AIM

To write a C program to implement the SCAN Disk Scheduling algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of disk requests.
Step 3 : Read the disk request sequence.
Step 4 : Read the initial head position.
Step 5 : Sort the disk requests in ascending order.
Step 6 : Move the disk head towards the higher track numbers, servicing all requests.
Step 7 : Reverse the direction after reaching the last request.
Step 8 : Calculate the total seek time.
Step 9 : Stop the program.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int req[20], n, head, i, j, temp;
   int seek = 0;
   printf("Enter number of requests: ");
   scanf("%d",&n);
   printf("Enter requests:\n");
   for(i=0;i<n;i++)
      scanf("%d",&req[i]);
   printf("Enter initial head position: ");
   scanf("%d",&head);
   /* Sort the requests */
   for(i=0;i<n-1;i++)
      for(j=i+1;j<n;j++)
         if(req[i]>req[j])
         {
            temp=req[i];
            req[i]=req[j];
            req[j]=temp;
         }
   /* Move towards higher tracks */
   for(i=0;i<n;i++)
   {
      if(req[i]>=head)
      {
         seek += abs(req[i]-head);
         head = req[i];
      }
   }
```

```
    /* Reverse direction */
    for(i=n-1;i>=0;i--){
        if(req[i]<head)
        {
            seek += abs(req[i]-head);
            head = req[i];
        }
    }
    printf("\nTotal Seek Time = %d",seek);
    return 0;
}
```

OUTPUT

```
Enter number of requests: 8

Enter requests:
98 183 37 122 14 124 65 67

Enter initial head position: 53

Total Seek Time = 299
```

RESULT:

Thus the SCAN Disk Scheduling algorithm was implemented successfully.

EX.NO : 11.4 C-SCAN DISK SCHEDULING ALGORITHM

DATE :

AIM

To write a C program to implement the Circular SCAN (C-SCAN) Disk Scheduling algorithm.

ALGORITHM

Step 1 : Start the program.
Step 2 : Read the number of disk requests.
Step 3 : Read the disk request sequence.
Step 4 : Read the initial head position.
Step 5 : Sort the disk requests in ascending order.
Step 6 : Move the disk head towards the higher track numbers and service all requests.
Step 7 : After reaching the last request, move the head to the beginning and continue servicing the remaining requests.
Step 8 : Calculate the total seek time.
Step 9 : Stop the program.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int req[20], n, head, i, j, temp;
   int seek = 0;
   printf("Enter number of requests: ");
   scanf("%d",&n);
   printf("Enter requests:\n");
   for(i=0;i<n;i++)
      scanf("%d",&req[i]);
   printf("Enter initial head position: ");
   scanf("%d",&head);
   /* Sort the requests */
   for(i=0;i<n-1;i++)
      for(j=i+1;j<n;j++)
         if(req[i]>req[j])
         {
            temp=req[i];
            req[i]=req[j];
            req[j]=temp;
         }
   /* Move towards higher tracks */
   for(i=0;i<n;i++)
   {
      if(req[i]>=head)
      {
         seek += abs(req[i]-head);
         head = req[i];
```

```
        }
    }
    /* Jump to the beginning */
    seek += head;
    head = 0;
    /* Continue servicing remaining requests */
    for(i=0;i<n;i++)
    {
        if(req[i]<head)
        {
            seek += abs(req[i]-head);
            head = req[i];
        }
    }
    printf("\nTotal Seek Time = %d",seek);
    return 0;
}
```

OUTPUT

```
Enter number of requests: 8
Enter requests:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Total Seek Time = 382
```

RESULT:

Thus the C-SCAN Disk Scheduling algorithm was implemented successfully.

EX.NO : 11.5 LOOK DISK SCHEDULING ALGORITHM
DATE :

AIM
To write a C program to implement the LOOK Disk Scheduling algorithm.

ALGORITHM
Step 1 : Start the program.
Step 2 : Read the number of disk requests.
Step 3 : Read the disk request sequence.
Step 4 : Read the initial head position.
Step 5 : Sort the disk requests in ascending order.
Step 6 : Move the disk head towards the higher track numbers and service all the requests.
Step 7 : Reverse the direction after servicing the last request.
Step 8 : Calculate the total seek time.
Step 9 : Stop the program.

PROGRAM
```
#include<stdio.h>
#include<stdlib.h>
int main()
{
   int req[20], n, head, i, j, temp;
   int seek = 0;
   printf("Enter number of requests: ");
   scanf("%d",&n);
   printf("Enter requests:\n");
   for(i=0;i<n;i++)
      scanf("%d",&req[i]);
   printf("Enter initial head position: ");
   scanf("%d",&head);
   /* Sort the requests */
   for(i=0;i<n-1;i++)
   {
      for(j=i+1;j<n;j++)
      {
         if(req[i]>req[j])
         {
            temp=req[i];
            req[i]=req[j];
            req[j]=temp;
         }
      }
   }
   /* Service requests greater than head */
   for(i=0;i<n;i++)
   {
      if(req[i]>=head)
```

```
        {
            seek += abs(req[i]-head);
            head=req[i];
        }
    }
    /* Reverse direction */
    for(i=n-1;i>=0;i--)
    {
        if(req[i]<head)
        {
            seek += abs(req[i]-head);
            head=req[i];
        }
    }
    printf("\nTotal Seek Time = %d",seek);
    return 0;
}
```

OUTPUT

```
Enter number of requests: 8
Enter requests:
98 183 37 122 14 124 65 67
Enter initial head position: 53
Total Seek Time = 299
```

RESULT:

Thus the LOOK Disk Scheduling algorithm was implemented successfully.